TRƯỜNG ĐẠI HỌC ĐIỆN LỰC

KHOA CÔNG NGHỆ THÔNG TIN

ĐẠI HỌC ĐIỆN LỰC
ELECTRIC POWER UNIVERSITY

BÁO CÁO CHUYÊN ĐỀ HỌC PHẦN

PHÁT TRIỂN PHẦN MỀM WEB AN TOÀN

ĐỀ TÀI:

PHÁT TRIỂN VÀ NÂNG CAO AN TOÀN BẢO MẬT CHO WEBSITE BÁN HÀNG THIẾT BỊ SỐ DDH ELECTRONICS

Giảng viên hướng dẫn: ThS. PHƯƠNG VĂN CẢNH

Sinh viên thực hiện : NGUYỄN VĂN ĐẠT

NGUYỄN VĂN HIẾU

NGUYỄN ĐỨC DƯƠNG

Ngành : CÔNG NGHỆ THÔNG TIN

Chuyên ngành : CÔNG NGHỆ PHẦN MỀM

Lớp : D18CNPM2

Khóa : 2023-2028

Hà Nội, ngày 06 tháng 01 năm 2027

PHIẾU CHẤM ĐIỂM

STT	Họ và tên sinh viên	Nội dung thực hiện	Điểm	Chữ ký
1	Nguyễn Văn Đạt (23810310093)	Cấu hình máy chủ Nginx HTTPS TLS 1.2/1.3, xây dựng bộ quy tắc phát hiện xâm nhập Snort IDS và giám sát an ninh thời gian thực.		
2	Nguyễn Văn Hiếu (23810310112)	Xây dựng kịch bản kiểm thử tấn công (Before vs After), tự động hóa kiểm thử an toàn hệ thống và tổng hợp viết báo cáo đồ án.		
3	Nguyễn Đức Dương (23810310091)	Quản lý dự án, gia cố an toàn mã nguồn Laravel theo chuẩn OWASP (chống SQL Injection, Brute Force, Upload Web Shell và Security Headers).		

Họ và tên giảng viên	Chữ ký	Ghi chú
Giảng viên chấm 1:		
Giảng viên chấm 2:		

MỤC LỤC

Trang

CHƯƠNG 1 GIỚI THIỆU CHUNG 3

1.1 Bối cảnh và lý do chọn đề tài 3

1.2 Mục tiêu nghiên cứu và phạm vi đề tài 4

1.2.1 Mục tiêu nghiên cứu 4

1.2.2 Phạm vi nghiên cứu 5

1.3 Tổng quan kiến trúc nền tảng công nghệ của hệ thống 5

1.3.1 Framework PHP Laravel 11 và mô hình kiến trúc MVC 5

1.3.2 Hệ quản trị cơ sở dữ liệu MySQL và cơ chế mã hóa 6

1.3.3 Máy chủ Web Reverse Proxy Nginx & Giao thức HTTPS 7

1.3.4 Hệ thống phát hiện xâm nhập mạng Snort IDS 8

1.3.5 Kiến trúc tổng thể mô hình Phòng thủ theo chiều sâu (Defense-in-Depth). 8

1.4 Phân tích các mối đe dọa an ninh theo tiêu chuẩn OWASP Top 10 9

CHƯƠNG 2 THIẾT KẾ GIẢI PHÁP KIẾN TRÚC AN TOÀN ĐA TẦNG 11

2.1 Phân tích mô hình Actor và Bề mặt xác thực hệ thống 11

2.1.1 Phân loại các Actor trong hệ thống 11

2.2 Thiết kế giải pháp bảo vệ tầng chu vi mạng & Máy chủ 12

2.2.1 Thiết kế máy chủ Reverse Proxy Nginx & Giao thức HTTPS 12

2.2.2 Thiết kế hệ thống phát hiện xâm nhập Snort IDS và tập luật my_sql.rules 13

2.3 Thiết kế kiến trúc an toàn tầng ứng dụng web (Laravel 11) 14

2.3.1 Thiết kế chuỗi trạm kiểm soát Middleware (Security Checkpoints) 14

2.3.2 Thiết kế ma trận phân quyền truy cập dựa trên vai trò (RBAC Matrix).... 15

2.3.3 Thiết kế luồng xác thực chống vét cạn (Rate Limiting & CAPTCHA Workflow)..... 16

2.3.4 Thiết kế quy trình tải tệp an toàn chống mã độc Web Shell (Secure Upload Workflow)..... 16

2.4 Giao diện sản phẩm 17

2.4.1 Giao diện Admin 18

2.4.2 Giao diện Khách hàng 21

CHƯƠNG 3 HIỆN THỰC HÓA VÀ GIA CỐ MÃ NGUỒN AN TOÀN 24

3.1 Gia cố phòng chống tấn công SQL Injection và trích xuất dữ liệu..... 24

3.1.1 Nguy cơ và kỹ thuật khai thác trước phòng thủ (Before) 24

3.1.2 Giải pháp gia cố bằng Eloquent Parameterized Binding (After) 24

3.2 Hiện thực hóa kiểm soát phân quyền quản trị (RBAC) 26

3.2.1 Nguy cơ leo thang đặc quyền trước phòng thủ (Before) 26

3.2.2 Cơ chế phân quyền đa tầng và mã lỗi 403 Forbidden (After) 27

3.3 Hiện thực hóa cơ chế chống tấn công vét cạn (Brute Force & CAPTCHA) 27

3.3.1 Nguy cơ trước phòng thủ (Before)..... 28

3.3.2 Cơ chế phòng ngự kép Rate Limiting và Math CAPTCHA (After) 28

3.4 Hiện thực hóa quy trình tải tệp tin an toàn chống Web Shell 29

3.4.1 Nguy cơ tải mã độc Web Shell trước phòng thủ (Before)..... 29

3.4.2 Thuật toán xử lý tệp tin an toàn đa lớp (After) 30

3.5 Thiết lập các tiêu đề an toàn trình duyệt (Security Headers Middleware) ... 30

3.5.1 Nguy cơ tấn công tầng trình duyệt trước phòng thủ (Before)..... 30

3.5.2 Giải pháp Middleware SecurityHeaders toàn cục (After)..... 31

3.6 Cấu hình bảo mật máy chủ Nginx HTTPS và hệ thống phát hiện xâm nhập Snort IDS..... 33

3.6.1 Cấu hình bảo mật máy chủ Nginx và mã hóa SSL/TLS 33

3.6.2 Cấu hình luật phát hiện xâm nhập trong Snort IDS (my_sql.rules)..... 34

CHƯƠNG 4 KỊCH BẢN THỰC NGHIỆM KIỂM THỬ VÀ ĐÁNH GIÁ HIỆU QUẢ AN TOÀN..... 36

4.1 Môi trường và công cụ thực nghiệm kiểm thử..... 36

4.1.1 Môi trường triển khai hệ thống 36

4.1.2 Công cụ kiểm thử và mô phỏng tấn công 36

4.2 Kịch bản kiểm thử các cuộc tấn công thực chiến (Before vs After) 37

4.2.1 Bài test 1: Kiểm thử tấn công SQL Injection cơ bản (Boolean-based)..... 37

4.2.2 Bài test 2: Kiểm thử tấn công trích xuất dữ liệu qua câu ghép (UNION SELECT)..... 38

4.2.3 Bài test 3: Dò quét và truy cập tệp tin cấu hình nhạy cảm (/.env)..... 39

4.2.4 Bài test 4: Tấn công vét cạn mật khẩu đăng nhập (Brute Force Login) 41

4.2.5 Bài test 5: Tấn công leo thang đặc quyền quản trị (Broken Access Control) 42

4.2.6 Bài test 6: Kiểm tra các tiêu đề bảo mật và an toàn phiên (Security Headers & Session) 43

4.3 Bảng tổng hợp đối chứng hiệu quả an toàn Trước và Sau phòng thủ..... 45

DANH MỤC HÌNH ẢNH

Trang

Hình 1.1 Kiến trúc mô hình MVC trong Laravel 11 6

Hình 1.2 sơ đồ mô hình phòng thủ theo chiều sâu (defense-in-depth) của hệ thống ddh-electronics 9

Hình 2.3 biểu đồ use case tổng quát của hệ thống 12

Hình 2.4 Giao diện Đăng nhập 17

Hình 2.5 Giao diện đăng kí 17

Hình 2.6 Giao diện Dashboard 18

Hình 2.7 Giao diện quản lý slide 18

Hình 2.8 Giao diện Quản lý người dùng 19

Hình 2.9 Giao diện Danh sách đơn hàng 19

Hình 2.10 Giao diện Quản lý sản phẩm 20

Hình 2.11 Giao diện Quản lý mã giảm giá 20

Hình 2.12 Giao diện Giỏ hàng 21

Hình 2.13 Giao diện đơn hàng khách hàng 21

Hình 2.14 Giao diện thanh toán 22

Hình 2.15 Giao diện đánh giá sản phẩm 22

Hình 2.16 Giao diện Chat box AI 23

Hình 2.17 Giao diện xem chi tiết sản phẩm 23

Hình 3.18 Đoạn mã ghép chuỗi SQL trực tiếp tiềm ẩn nguy cơ SQL Injection. 24

Hình 3.19 Truy vấn tham số hóa (Prepared Statements) trong ProductController 25

Hình 3.20 Truy vấn tham số hóa bảo vệ API trong MobileApiController 25

Hình 3.21 Cấu hình Middleware kiểm soát phân quyền RBAC và mã lỗi 403 Forbidden trong AdminController 27

Hình 3.22 Cấu hình Rate Limiting giới hạn tần suất đăng nhập trong routes/web.php 28

Hình 3.23 Thuật toán sinh mã bảo vệ Math CAPTCHA động trong LoginController 29

Hình 3.24 Hàm xử lý tải tệp tin an toàn đa lớp handleSecureImageUpload trong AdminController 30

Hình 3.25 Mã nguồn Middleware SecurityHeaders thiết lập các tiêu đề an toàn OWASP 31

Hình 3.26 Đăng ký Middleware SecurityHeaders vào nhân ứng dụng tại bootstrap/app.php 32

Hình 3.27 Cấu hình Nginx Virtual Host bảo mật và mã hóa SSL/TLS trong ddh-electronics.conf 34

Hình 3.28 Bộ quy tắc phát hiện xâm nhập my_sql.rules trong hệ thống Snort IDS 35

Hình 4.29 Kết quả kiểm thử tấn công SQL Injection cơ bản trên giao diện và cảnh báo thời gian thực của Snort IDS 38

Hình 4.30 Cảnh báo thời gian thực của hệ thống Snort IDS khi phát hiện tấn công trích xuất dữ liệu UNION SELECT 39

Hình 4.31 Cảnh báo thời gian thực của hệ thống Snort IDS khi phát hiện hành vi dò quét tệp tin nhạy cảm .env 40

Hình 4.32 Cảnh báo thời gian thực của Snort IDS khi phát hiện tấn công vét cạn mật khẩu Brute Force 42

Hình 4.33 Phản hồi ngăn chặn truy cập trái phép bằng mã lỗi HTTP 403 Forbidden 43

Hình 4.34 Kiểm tra các tiêu đề bảo mật HTTP và cờ Session Cookie qua DevTools 44

DANH MỤC BẢNG

Bảng 1.1 Bảng phân cấp cơ chế lưu trữ và bảo mật dữ liệu trong MySQL..... 7

Bảng 1.2 So sánh mức độ an toàn giữa giao thức HTTP truyền thống và HTTPS (TLS 1.2/1.3) 7

Bảng 1.3 Bảng phân tích các nguy cơ tấn công trọng yếu vào hệ thống DDH Electronics 9

Bảng 2.4 Bảng thiết kế các quy tắc phát hiện xâm nhập trong Snort IDS (my_sql.rules) 13

Bảng 2.5 Ma trận phân quyền truy cập tài nguyên hệ thống (RBAC)..... 15

Bảng 3.6 Bảng phân tích chức năng các tiêu đề bảo mật HTTP được áp dụng . 32

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa và chuyển đổi số hiện nay, Thương mại điện tử (E-Commerce) đã trở thành một phần không thể thiếu trong hoạt động sản xuất kinh doanh và đời sống xã hội. Cùng với sự phát triển mạnh mẽ về quy mô và doanh thu, các hệ thống ứng dụng web thương mại điện tử cũng trở thành mục tiêu hàng đầu của các cuộc tấn công mạng nguy hiểm, từ việc đánh cắp thông tin tài chính của khách hàng, chiếm đoạt tài khoản quản trị cho đến phá hoại hoặc chiếm quyền điều khiển máy chủ.

Theo thống kê thường niên của Dự án Bảo mật Ứng dụng Web Mở (**OWASP**), các lỗ hổng an ninh kinh điển như Tiêm mã độc (**SQL Injection**), Kiểm soát phân quyền lỏng lẻo (**Broken Access Control**), Tấn công vét cạn mật khẩu (**Brute Force**), và Tải lên tệp tin độc hại (**Unrestricted File Upload**) vẫn đang chiếm tỷ lệ rất cao và gây ra những thiệt hại nghiêm trọng. Do đó, yêu cầu đặt ra đối với một kỹ sư phần mềm hiện đại không chỉ dừng lại ở việc xây dựng một hệ thống hoạt động trơn tru về mặt nghiệp vụ, mà quan trọng hơn cả là phải tích hợp tư duy an toàn bảo mật ngay từ những bước đầu tiên của vòng đời phát triển phần mềm (**DevSecOps**).

Nhận thức rõ tầm quan trọng sống còn của vấn đề trên, trong khuôn khổ học phần **Phát triển phần mềm web an toàn** do **ThS. Phương Văn Cảnh** giảng dạy, nhóm sinh viên chúng em đã lựa chọn đề tài: **"Phát triển và nâng cao an toàn bảo mật cho website bán hàng thiết bị số DDH Electronics"**.

Đề tài được xây dựng theo mô hình **Phòng thủ theo chiều sâu (Defense-in-Depth)**, bao quát toàn diện từ tầng mã nguồn ứng dụng đến tầng hạ tầng mạng:

1. **Tầng ứng dụng (Application Layer):** Gia cố toàn diện mã nguồn Laravel 11 theo chuẩn OWASP Top 10, triển khai cơ chế tham số hóa dữ liệu (Prepared Statements) chống SQLi, xác thực đa lớp với Rate Limiting & CAPTCHA chống Brute Force, phòng chống Web Shell bằng phân tích nội dung nhị phân (Finfo MIME) và kiểm soát truy cập quản trị bằng ma trận phân quyền (RBAC).
2. **Tầng chu vi & mạng (Perimeter & Network Layer):** Thiết lập máy chủ Nginx Virtual Host với chuẩn mã hóa an toàn HTTPS (TLS 1.2/1.3), che

giấu tệp tin cấu hình nhạy cảm (.env), đồng thời tích hợp hệ thống phát hiện xâm nhập **Snort IDS** giám sát và cảnh báo các hành vi tấn công thời gian thực.

3. **Tầng thực nghiệm (Testing Layer):** Xây dựng các kịch bản kiểm thử thâm nhập đối chứng (**Before vs After**) nhằm chứng minh tính hiệu quả vượt trội của các giải pháp phòng thủ đã được áp dụng.

Nhóm chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất tới **ThS. Phương Văn Cảnh**, giảng viên bộ môn, người đã tận tình hướng dẫn, truyền đạt những kiến thức chuyên môn quý báu và định hướng giải pháp kỹ thuật thực tiễn cho chúng em trong suốt quá trình học tập và hoàn thiện đồ án này.

Dù đã có nhiều cố gắng, song do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo đồ án khó tránh khỏi những thiếu sót nhất định. Nhóm chúng em rất mong nhận được những ý kiến đóng góp quý báu từ thầy để đề tài ngày càng được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Hà Nội, ngày 06 tháng 01 năm 2027

Nhóm sinh viên thực hiện

Nguyễn Đức Dương – Nguyễn Văn Đạt – Nguyễn Văn Hiếu

CHƯƠNG 1 GIỚI THIỆU CHUNG

1.1 Bối cảnh và lý do chọn đề tài

Trong bối cảnh nền kinh tế số phát triển vượt bậc, thương mại điện tử (E-Commerce) đã trở thành một kênh giao thương trọng yếu của các doanh nghiệp bán lẻ. Tuy nhiên, cùng với lượng giao dịch và người dùng tăng vọt, các website thương mại điện tử cũng trở thành đích ngắm ưa thích của giới tội phạm mạng. Các hệ thống này luôn chứa đựng những nguồn dữ liệu vô cùng giá trị: thông tin định danh cá nhân (PII), lịch sử mua sắm, cơ sở dữ liệu sản phẩm, doanh thu và đặc biệt là các cổng thanh toán trực tuyến.

Thực tế tại Việt Nam và trên thế giới cho thấy, phần lớn các ứng dụng web thương mại điện tử hiện nay vẫn tồn tại nhiều lỗ hổng bảo mật nghiêm trọng do quy trình phát triển phần mềm chỉ tập trung vào việc hoàn thiện tính năng nghiệp vụ mà xem nhẹ các nguyên tắc lập trình an toàn (**Secure Coding Standards**). Kẻ tấn công có thể dễ dàng khai thác các kỹ thuật:

- **Tiêm mã độc (SQL Injection):** Vượt qua bước đăng nhập quản trị hoặc trích xuất toàn bộ dữ liệu bảng tài khoản và mật khẩu người dùng.
- **Tấn công vét cạn (Brute Force Attack):** Sử dụng các kịch bản tự động (Botnet/Script) gửi hàng nghìn yêu cầu đăng nhập liên tục để dò quét mật khẩu quản trị viên.
- **Tải lên tệp tin độc hại (Unrestricted File Upload):** Ngụy trang mã độc dưới dạng tệp tin hình ảnh sản phẩm để đẩy lên máy chủ mã độc Web Shell (như PHP backdoor), từ đó chiếm quyền điều khiển toàn bộ máy chủ (Remote Code Execution - RCE).
- **Kiểm soát phân quyền lỏng lẻo (Broken Access Control):** Khách hàng thông thường sau khi đăng nhập có thể tùy tiện truy cập vào các đường dẫn nội bộ dành riêng cho Quản trị viên (/admin) để xem đơn hàng hoặc thay đổi giá sản phẩm.

- **Lộ lọt thông tin nhạy cảm qua tệp tin cấu hình:** Quản trị viên cấu hình máy chủ Web thiếu an toàn để lộ tệp tin .env chứa mật khẩu kết nối cơ sở dữ liệu và khóa bí mật của ứng dụng.

Xuất phát từ yêu cầu thực tiễn đó, nhóm đã lựa chọn đề tài **"Phát triển và nâng cao an toàn bảo mật cho website bán hàng thiết bị số DDH Electronics"**. Đề tài lấy website DDH Electronics (được xây dựng trên nền tảng PHP Laravel 11 hiện đại) làm môi trường thực nghiệm, từ đó nghiên cứu, áp dụng và kiểm chứng toàn diện các giải pháp phòng thủ đa tầng từ mã nguồn ứng dụng cho đến hạ tầng mạng.

1.2 Mục tiêu nghiên cứu và phạm vi đề tài

1.2.1 Mục tiêu nghiên cứu

Đề tài hướng tới 3 mục tiêu cốt lõi:

1. **Bảo vệ mã nguồn ứng dụng (Application Hardening):** Triển khai các biện pháp kỹ thuật lập trình an toàn theo khuyến nghị của tổ chức bảo mật quốc tế **OWASP (Open Web Application Security Project)** nhằm triệt tiêu hoàn toàn các lỗ hổng phổ biến (SQLi, Brute Force, Web Shell, Broken Access Control, Clickjacking, MIME Sniffing).
2. **Thiết lập chu vi phòng thủ hạ tầng mạng (Perimeter Defense):** Triển khai giao thức truyền thông an toàn **HTTPS (TLS 1.2/1.3)** qua máy chủ Web Nginx, đồng thời tích hợp hệ thống phát hiện xâm nhập **Snort IDS** với bộ luật tùy biến (my_sql.rules) để phát hiện, giám sát và phát cảnh báo sớm các hành vi thâm nhập trái phép theo thời gian thực.
3. **Thực nghiệm và đánh giá đối chứng (Verification & Penetration Testing):** Xây dựng bộ công cụ kiểm thử tự động, đóng vai trò của kiểm thử viên thâm nhập (Penetration Tester) để kiểm chứng hệ thống qua mô hình **Before vs After** (Trước khi bảo mật vs Sau khi bảo mật).

1.2.2 Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Nền tảng ứng dụng web thương mại điện tử DDH Electronics viết trên framework Laravel 11, hệ quản trị cơ sở dữ liệu MySQL, máy chủ Reverse Proxy Nginx và hệ thống giám sát an ninh mạng Snort IDS.
- **Môi trường thực nghiệm:** Hệ thống cục bộ (Localhost) kết hợp đường hầm bảo mật công khai **Cloudflare Tunnel (HTTPS)** phục vụ việc đánh giá và kiểm thử từ xa.

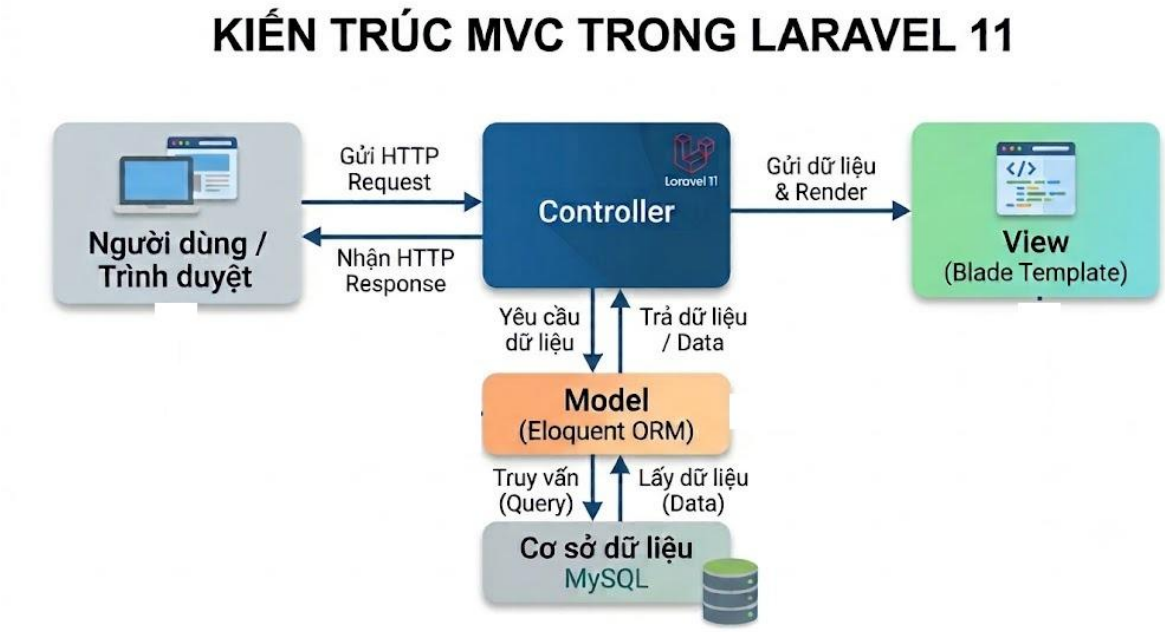
1.3 Tổng quan kiến trúc nền tảng công nghệ của hệ thống

Hệ thống DDH Electronics được thiết kế và vận hành dựa trên sự kết hợp đồng bộ giữa tầng ứng dụng hiện đại và các thành phần bảo vệ hạ tầng mạng:

1.3.1 Framework PHP Laravel 11 và mô hình kiến trúc MVC

Laravel là một trong những PHP Framework phổ biến và mạnh mẽ nhất thế giới hiện nay, áp dụng kiến trúc **Model - View - Controller (MVC)** giúp tách bạch giữa tầng xử lý nghiệp vụ, giao diện người dùng và cơ sở dữ liệu:

- **Model (Eloquent ORM):** Quản lý cấu trúc dữ liệu và quan hệ giữa các bảng. Điểm nổi bật nhất của Eloquent là mặc định sử dụng cơ chế **PDO Parameterized Binding**, tự động tham số hóa mọi giá trị đầu vào để ngăn chặn triệt để lỗ hổng SQL Injection.
- **View (Blade Template Engine):** Xử lý giao diện người dùng. Cú pháp `{{ $variable }}` của Blade tự động áp dụng hàm `htmlspecialchars()` của PHP, giúp mã hóa các ký tự đặc biệt (<, >, &, ", ') để phòng chống tấn công XSS (Cross-Site Scripting).
- **Controller & Middleware Pipeline:** Điều phối luồng dữ liệu của ứng dụng. Cơ chế Middleware cho phép hệ thống tạo ra một chuỗi các "trạm kiểm soát" an ninh (Security Checkpoints) trước khi yêu cầu của người dùng chạm tới Controller (ví dụ: kiểm tra đăng nhập `auth`, phân quyền quản trị `is_admin`, chống vét cạn `throttle:5,0.33`, và gắn tiêu đề bảo vệ trình duyệt `SecurityHeaders`).



Hình 1.1 Kiến trúc mô hình MVC trong Laravel 11

1.3.2 Hệ quản trị cơ sở dữ liệu MySQL và cơ chế mã hóa

Hệ thống sử dụng MySQL làm nơi lưu trữ toàn bộ dữ liệu người dùng, danh mục sản phẩm, đơn hàng và các bảng phân quyền:

- **Mã hóa mật khẩu bằng Bcrypt:** Mọi mật khẩu của khách hàng và quản trị viên đều được băm một chiều bằng thuật toán **Bcrypt** kèm chuỗi muối (Salt) ngẫu nhiên, đảm bảo chống lại các cuộc tấn công tra cứu bảng Rainbow Table.
- **Bảo vệ tính toàn vẹn dữ liệu:** Toàn bộ bảng dữ liệu được thiết lập chuẩn bảng mã utf8mb4_unicode_ci giúp hiển thị tiếng Việt chính xác và loại bỏ các ký tự rác độc hại.

Bảng 1.1 Bảng phân cấp cơ chế lưu trữ và bảo mật dữ liệu trong MySQL

Loại dữ liệu	Trường dữ liệu	Phương thức bảo mật áp dụng	Mục tiêu bảo vệ
Mật khẩu tài khoản	users.password	Thuật toán băm một chiều Bcrypt (Salt tự động)	Chống lộ mật khẩu gốc khi bị trích xuất CSDL
Dữ liệu người dùng	email, phone, address	Parameterized Binding (PDO) & Validation	Chống SQL Injection và làm sai lệch dữ liệu
Phân quyền vai trò	users.is_admin	Cột boolean nhị phân (0 hoặc 1)	Xác thực quyền quản trị viên, chống leo thang đặc quyền
Đơn hàng & Giao dịch	orders, order_items	Ràng buộc khóa ngoại (Foreign Keys) & Transaction	Chống can thiệp sửa đổi giá trị đơn hàng

1.3.3 Máy chủ Web Reverse Proxy Nginx & Giao thức HTTPS

Nginx đóng vai trò là tầng tiếp nhận đầu tiên (Reverse Proxy) đón nhận các yêu cầu mạng từ Internet trước khi chuyển tiếp về ứng dụng nội bộ:

- **Mã hóa đường truyền bằng HTTPS (TLS 1.2/1.3):** Bảo vệ toàn bộ dữ liệu trao đổi giữa trình duyệt của khách hàng và máy chủ, ngăn chặn kẻ tấn công trên cùng mạng Wi-Fi nghe lén hoặc đánh cắp Cookie phiên làm việc (Man-in-the-Middle Attack).
- **Chặn tệp tin nhạy cảm:** Nginx được cấu hình quy tắc từ chối truy cập toàn bộ các tệp tin và thư mục ẩn bắt đầu bằng dấu chấm (.env, .git), cô lập hoàn toàn Document Root tại thư mục /public.

Bảng 1.2 So sánh mức độ an toàn giữa giao thức HTTP truyền thống và HTTPS (TLS 1.2/1.3)

Tiêu chí đánh giá	Giao thức HTTP truyền thống	Giao thức HTTPS (TLS 1.2/1.3) trên Nginx
Cổng kết nối mặc định	Cổng 80 (Không mã hóa)	Cổng 443 (Mã hóa SSL/TLS)
Hình thức truyền dữ liệu	Văn bản thuần (Plain Text)	Bản mã nhị phân (Encrypted Ciphertext)

Tiêu chí đánh giá	Giao thức HTTP truyền thống	Giao thức HTTPS (TLS 1.2/1.3) trên Nginx
Nguy cơ nghe lén (Sniffing)	Rất cao, lộ tài khoản và thông tin thẻ	Bị triệt tiêu hoàn toàn nhờ khóa phiên đối xứng
Chuyển hướng an toàn	Cho phép truy cập không an toàn	Tự động chuyển hướng cưỡng chế (301 Redirect sang HTTPS)

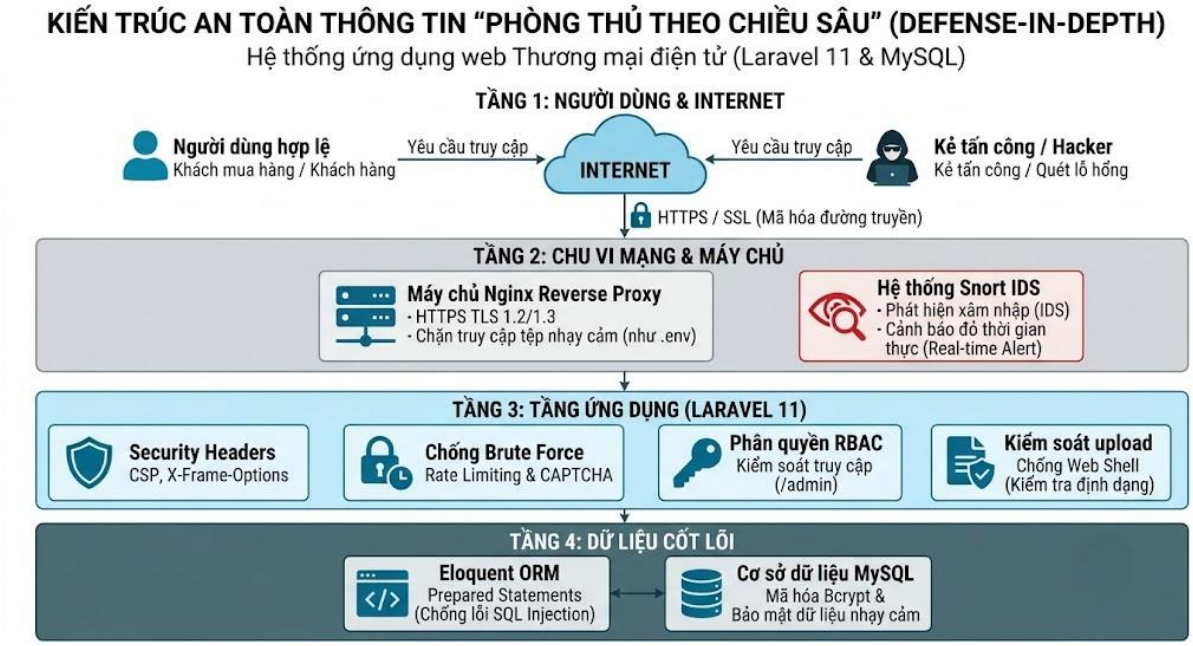
1.3.4 Hệ thống phát hiện xâm nhập mạng Snort IDS

Snort là hệ thống phát hiện và ngăn ngừa xâm nhập mạng (NIDS/NIPS) mã nguồn mở hàng đầu thế giới. Trong đề tài này, Snort đóng vai trò như một "người gác cổng chu vi", liên tục lắng nghe và phân tích cấu trúc từng gói tin TCP/IP đi qua các cổng dịch vụ web (Port 80, 8000, 443):

- **Cơ chế hoạt động:** So khớp chuỗi dữ liệu (Deep Packet Inspection) dựa trên bộ quy tắc chuyên biệt `my_sql.rules`.
- **Khả năng phản ứng:** Khi phát hiện gói tin mang dấu hiệu tấn công (chẳng hạn như mệnh đề `' OR 1=1` hoặc `UNION SELECT`), hệ thống lập tức ghi nhật ký an ninh (Log) và phát cảnh báo trực quan theo thời gian thực tới người quản trị.

1.3.5 Kiến trúc tổng thể mô hình Phòng thủ theo chiều sâu (Defense-in-Depth)

Nhằm bảo vệ hệ thống trước các kỹ thuật tấn công đa tầng, nhóm đã kết hợp đồng bộ 3 lớp rào chắn liên hoàn: Tầng chu vi mạng (Snort IDS) -> Tầng máy chủ (Nginx HTTPS) -> Tầng ứng dụng lõi (Laravel 11).



Hình 1.2 sơ đồ mô hình phòng thủ theo chiều sâu (defense-in-depth) của hệ thống ddh-electronics

1.4 Phân tích các mối đe dọa an ninh theo tiêu chuẩn OWASP Top 10

Tổ chức **OWASP (Open Web Application Security Project)** là cộng đồng quốc tế uy tín hàng đầu chuyên đưa ra bảng xếp hạng 10 nguy cơ an ninh nghiêm trọng nhất đối với ứng dụng web. Dựa trên danh mục **OWASP Top 10 (phiên bản 2021)**, nhóm đã xác định 5 bề mặt tấn công trọng điểm cần gia cố trên hệ thống DDH Electronics:

Bảng 1.3 Bảng phân tích các nguy cơ tấn công trọng yếu vào hệ thống DDH Electronics

Mã OWASP	Tên lỗ hổng an ninh	Kỹ thuật tấn công của Hacker	Mức độ rủi ro & Hậu quả
A01:2021	**Broken Access Control** (Kiểm soát truy cập lỏng lẻo)	Khách hàng thông thường cố tình truy cập vào URL /admin hoặc thay đổi ID đơn hàng của người khác trên	**Rất cao:** Lộ toàn bộ thông tin khách hàng, doanh thu và can thiệp trái phép vào hệ thống quản trị.

Mã OWASP	Tên lỗ hổng an ninh	Kỹ thuật tấn công của Hacker	Mức độ rủi ro & Hậu quả
		URL để xem trộm thông tin.	
A03:2021	**Injection** (Tiêm mã độc SQL Injection)	Chèn các chuỗi SQL độc hại như ' OR 1=1 -- hoặc UNION SELECT vào ô tìm kiếm sản phẩm hoặc form đăng nhập.	**Nghiêm trọng:** Vượt qua xác thực đăng nhập, trích xuất dữ liệu nhạy cảm hoặc phá hủy bảng dữ liệu (DROP TABLE).
A04:2021	**Insecure Design / Unrestricted Upload** (Tải file độc hại)	Ngụy trang mã nguồn thực thi độc hại (file PHP Web Shell) thành file ảnh rồi tải lên qua chức năng cập nhật ảnh đại diện/ảnh sản phẩm.	**Đặc biệt nghiêm trọng:** Chiếm quyền thực thi mã từ xa (RCE), kiểm soát toàn bộ máy chủ Web.
A05:2021	**Security Misconfiguration** (Sai sót cấu hình an ninh)	Dò quét các tệp tin cấu hình nhạy cảm (/.env, /.git) do máy chủ không chặn quyền truy cập; thiếu các HTTP Security Headers.	**Cao:** Lộ mật khẩu cơ sở dữ liệu (DB_PASSWORD), lộ khóa ứng dụng (APP_KEY), bị tấn công Clickjacking lồng iframe.
A07:2021	**Identification & Authentication Failures** (Lỗi định danh & xác thực)	Sử dụng công cụ tự động gửi hàng loạt request POST vào /login (tấn công vét cạn Brute Force) để bẻ khóa mật khẩu admin.	**Cao:** Tài khoản quản trị bị chiếm đoạt, tài nguyên máy chủ bị nghẽn do quá tải request rác.

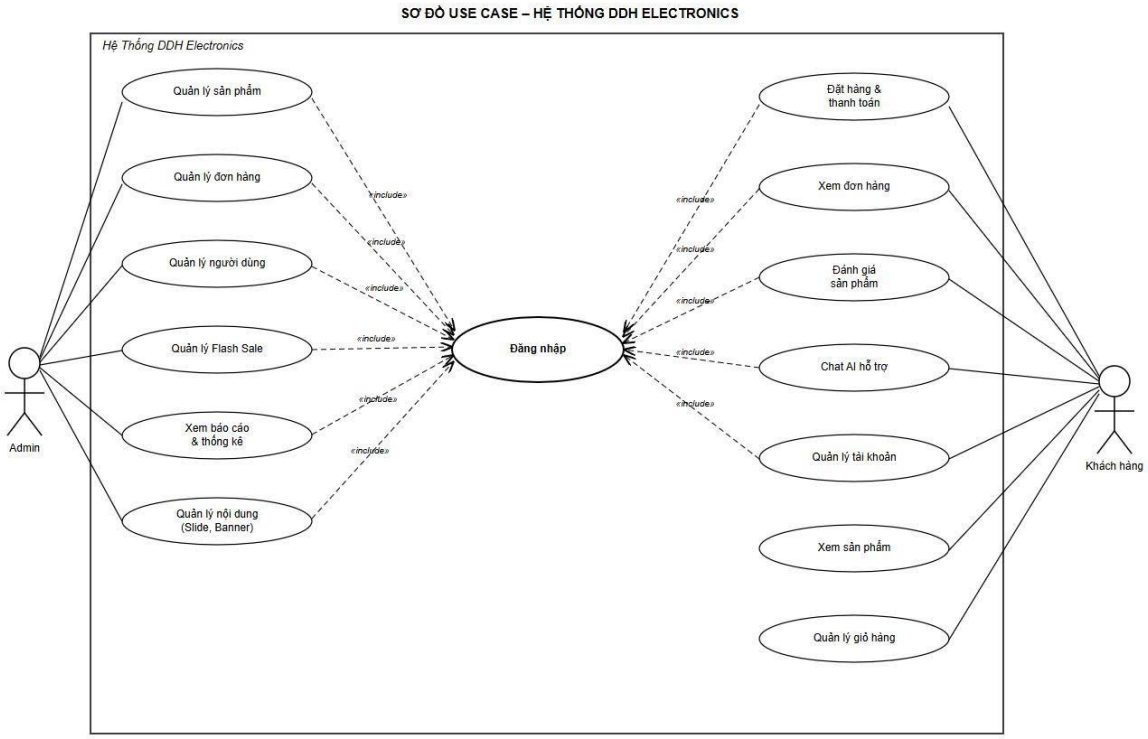
CHƯƠNG 2 THIẾT KẾ GIẢI PHÁP KIẾN TRÚC AN TOÀN ĐA TẦNG

2.1 Phân tích mô hình Actor và Bề mặt xác thực hệ thống

Để thiết kế kiến trúc bảo mật chính xác, trước tiên cần phân loại rõ ràng các tác nhân (Actors) tham gia tương tác với hệ thống DDH Electronics và ranh giới quyền hạn của từng tác nhân:

2.1.1 Phân loại các Actor trong hệ thống

- **Khách vãng lai (Guest):** Người dùng chưa đăng nhập, chỉ có quyền duyệt xem sản phẩm, bài viết tin tức, tìm kiếm và thêm hàng vào giỏ tạm thời.
- **Khách hàng (Customer / User):** Người dùng đã đăng nhập hợp lệ (is_admin = 0), có quyền quản lý thông tin cá nhân, đặt hàng, thanh toán trực tuyến, theo dõi trạng thái đơn hàng của chính mình và viết đánh giá sản phẩm.
- **Quản trị viên (Administrator):** Tài khoản quản trị cấp cao (is_admin = 1), toàn quyền truy cập khu vực /admin để quản lý sản phẩm, danh mục, đơn hàng, người dùng, cấu hình khuyến mãi và thống kê doanh thu.
- **Tác nhân đe dọa (Attacker / Pentester):** Kẻ tấn công hoặc kiểm thử viên bên ngoài cố tình gửi các gói tin thăm dò, rà quét cổng, tiêm mã SQL Injection hoặc vét cạn mật khẩu nhằm chiếm đoạt tài nguyên.



Hình 2.3 biểu đồ use case tổng quát của hệ thống

2.2 Thiết kế giải pháp bảo vệ tầng chu vi mạng & Máy chủ

Hạ tầng mạng là tuyến phòng thủ đầu tiên ngăn chặn các cuộc tấn công từ bên ngoài trước khi gói tin chạm tới tầng mã nguồn Laravel.

2.2.1 Thiết kế máy chủ Reverse Proxy Nginx & Giao thức HTTPS

- **Cơ chế hoạt động:** Nginx tiếp nhận toàn bộ lưu lượng cổng 80 và cổng 443 từ Internet. Tại đây, Nginx đóng vai trò "lá chắn che giấu" (Shielding), giấu hoàn toàn địa chỉ IP và cấu trúc nội bộ của máy chủ backend.
- **Cưỡng chế chuyển hướng (301 Permanent Redirect):** Mọi request gửi qua giao thức HTTP không an toàn (Port 80) lập tức bị cưỡng chế chuyển hướng 301 sang giao thức bảo mật HTTPS (Port 443).
- **Thiết kế chặn tệp tin nhạy cảm:** Áp dụng khối điều hướng `location ~ /\. { deny all; }` nhằm chặn đứng tuyệt đối mọi hành vi dò quét các tệp tin cấu hình (.env, .git, .htaccess).

2.2.2 Thiết kế hệ thống phát hiện xâm nhập Snort IDS và tập luật my_sql.rules

Snort IDS được thiết lập ở chế độ giám sát mạng theo thời gian thực (Network Intrusion Detection System). Nhóm đã thiết kế riêng bộ quy tắc chuyên biệt my_sql.rules gồm 5 quy tắc an ninh tương ứng với các mối đe dọa thực tế:

Bảng 2.4 Bảng thiết kế các quy tắc phát hiện xâm nhập trong Snort IDS (my_sql.rules)

Mã luật (SID)	Tên quy tắc & Mẫu nhận diện (Signature)	Mức độ cảnh báo	Ý nghĩa kỹ thuật & Mục tiêu phát hiện
100001	content:'' OR 1=1'' (nocase)	Priority 1 (Cao nhất)	Phát hiện hành vi tiêm mệnh đề boolean luôn đúng ' OR 1=1 nhằm vượt qua cơ chế xác thực hoặc bypass logic tìm kiếm.
100002	content:"UNION" + content:"SELECT" (nocase)	Priority 1 (Cao nhất)	Phát hiện kỹ thuật tấn công UNION-based SQLi nhằm trích xuất trái phép dữ liệu tài khoản và mật khẩu từ bảng users.
100003	content:"DROP TABLE" (nocase)	Priority 2 (Trung bình)	Phát hiện hành vi phá hoại dữ liệu có chủ đích bằng câu lệnh xóa sổ toàn bộ bảng dữ liệu trong hệ thống.
100004	content:".env" hoặc content:"wp-admin" (nocase)		Phát hiện hành vi dùng công cụ rà quét (Scanner) thăm dò tệp tin cấu hình

Mã luật (SID)	Tên quy tắc & Mẫu nhận diện (Signature)	Mức độ cảnh báo	Ý nghĩa kỹ thuật & Mục tiêu phát hiện
			nhạy cảm và các đường dẫn quản trị.
100005	threshold: type both, track by_src, count 5, seconds 20		Phát hiện hành vi gửi liên tiếp từ 5 request POST vào /login trong vòng 20 giây (Dấu hiệu của tấn công vét cạn mật khẩu Brute Force).

2.3 Thiết kế kiến trúc an toàn tầng ứng dụng web (Laravel 11)

Tầng ứng dụng là nơi trực tiếp tiếp nhận và xử lý dữ liệu người dùng. Nhóm đã thiết kế các cơ chế phòng thủ chuyên sâu nhằm bảo đảm an toàn dữ liệu:

2.3.1 Thiết kế chuỗi trạm kiểm soát Middleware (Security Checkpoints)

Toàn bộ các yêu cầu HTTP gửi đến website trước khi được đưa vào xử lý tại Controller đều phải đi qua chuỗi Middleware bảo vệ tuần tự:

- **Trạm 1 - SecurityHeaders:** Tự động gắn các cờ bảo vệ trình duyệt (CSP, X-Frame-Options, nosniff, HSTS) vào toàn bộ phản hồi HTTP.
- **Trạm 2 - SnortIdSimulator:** Soi quét payload yêu cầu và kích hoạt cảnh báo an ninh mạng.
- **Trạm 3 - RateLimiter (throttle:5,0.33):** Đếm tần suất yêu cầu đăng nhập, ngắt kết nối tạm thời nếu vượt quá ngưỡng cho phép.
- **Trạm 4 - Authentication (auth) & Authorization (RBAC):** Kiểm tra trạng thái đăng nhập và vai trò quản trị viên trước khi mở quyền vào khu vực nhạy cảm.

2.3.2 Thiết kế ma trận phân quyền truy cập dựa trên vai trò (RBAC Matrix)

Nhằm triệt tiêu hoàn toàn lỗ hổng OWASP A01: Broken Access Control, hệ thống thiết lập ma trận kiểm soát truy cập phân định rạch ròi giữa 3 nhóm người dùng:

Bảng 2.5 Ma trận phân quyền truy cập tài nguyên hệ thống (RBAC)

Chức năng / Tài nguyên hệ thống	**Khách vãng lai (Guest)**	**Khách hàng (Customer)**	**Quản trị viên (Admin)**	**Mã phản hồi khi vi phạm**
Xem danh sách sản phẩm, tin tức	Cho phép	Cho phép	Cho phép	200 OK
Thêm vào giỏ hàng, áp mã giảm giá	Cho phép	Cho phép	Cho phép	200 OK
Đặt hàng và Thanh toán (VNPay/MoMo)	Bắt đăng nhập	Cho phép	Cho phép	302 Chuyển hướng /login
Quản lý thông tin cá nhân, lịch sử đơn	Từ chối	Cho phép	Cho phép	302 Chuyển hướng /login
Truy cập Dashboard quản trị (/admin)	Từ chối	Bị cấm	Cho phép	403 Forbidden
Quản lý sản phẩm, đơn hàng, danh mụ	Từ chối	Bị cấm	Cho phép	403 Forbidden
Bật/Tắt Flash Sale, Thống kê doanh thu	Từ chối	Bị cấm	Cho phép	403 Forbidden

2.3.3 Thiết kế luồng xác thực chống vét cạn (Rate Limiting & CAPTCHA Workflow)

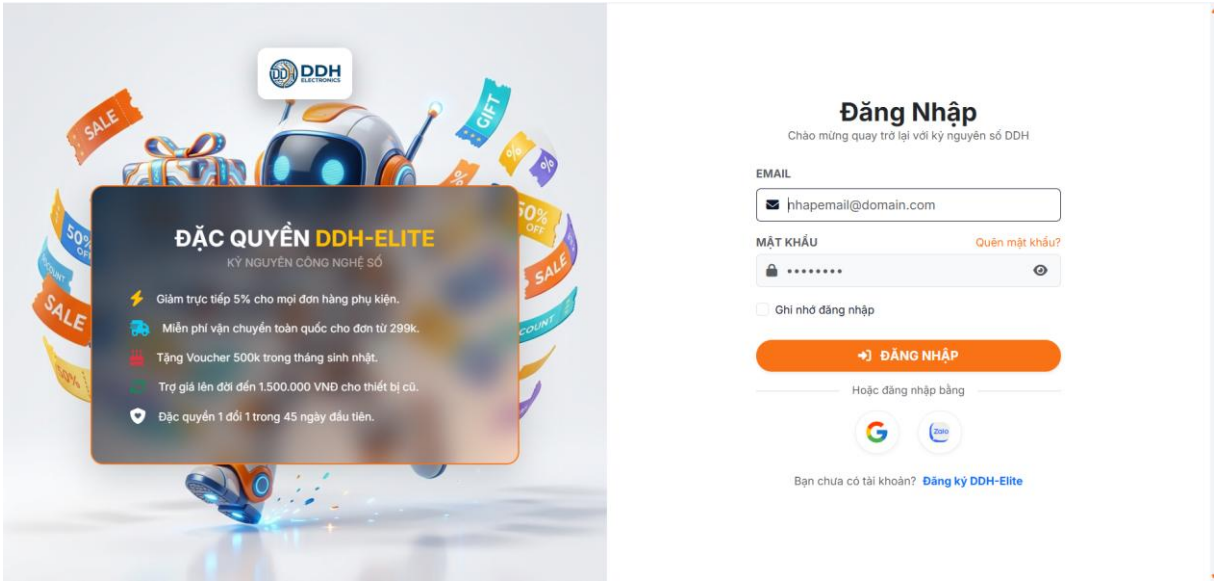
- **Thuật toán Token Bucket (Rate Limiting):** Cấu hình giới hạn tối đa 5 lần gửi yêu cầu POST đăng nhập trong vòng 20 giây cho mỗi địa chỉ IP. Khi vượt ngưỡng, hệ thống lập tức trả về mã lỗi 429 Too Many Requests.
- **Cơ chế xác thực người thật (Math CAPTCHA):** Mỗi phiên đăng nhập sinh ngẫu nhiên một phép tính số học (ví dụ: 4 + 5 = ?) lưu tại Session phía Server. Kẻ tấn công dùng kịch bản tự động sẽ không thể bypass qua bước xác thực này nếu không giải đúng biểu thức toán học.

2.3.4 Thiết kế quy trình tải tệp an toàn chống mã độc Web Shell (Secure Upload Workflow)

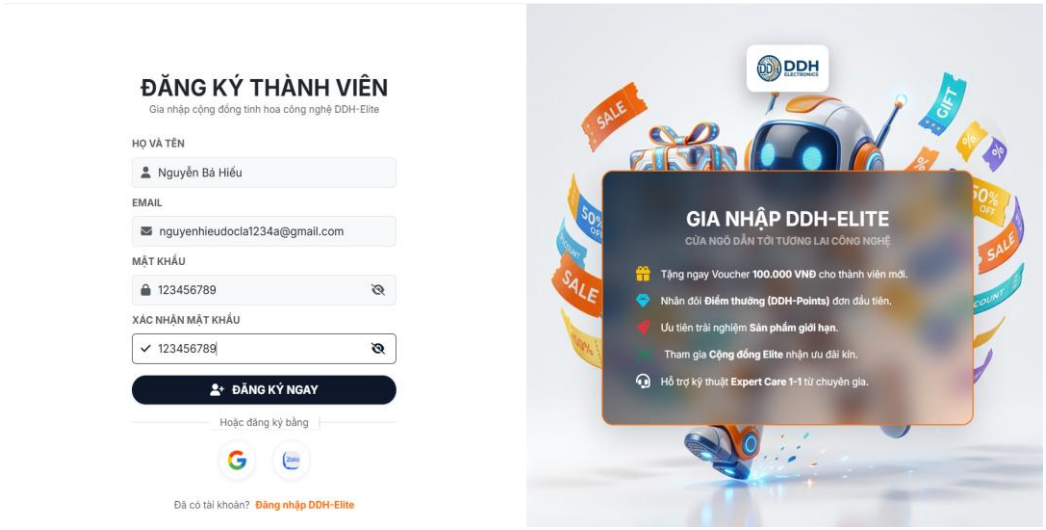
Chức năng tải ảnh đại diện và ảnh sản phẩm được thiết kế theo quy trình **phòng ngự 4 bước khép kín**:

1. **Kiểm tra phần mở rộng (Blacklist & Whitelist):** Từ chối ngay lập tức các tệp có đuôi thực thi nguy hiểm (.php, .phtml, .exe, .sh).
2. **Kiểm tra chữ ký nhị phân (Binary Magic Bytes / MIME Finfo):** Dùng thư viện finfo đọc cấu trúc nhị phân thực tế của tệp, ngăn chặn kỹ thuật đổi tên file độc hại thành shell.php.jpg.
3. **Đổi tên tệp ngẫu nhiên bằng UUID v4:** Tên tệp được sinh ngẫu nhiên dưới dạng mã định danh 36 ký tự (ví dụ: a3f1c2d4-....png), triệt tiêu hoàn toàn kỹ thuật tấn công duyệt thư mục (Path Traversal ../../).
4. **Lưu trữ cách ly:** Tệp tin được đưa vào thư mục lưu trữ tĩnh được cấu hình cấm thực thi mã script.

2.4 Giao diện sản phẩm

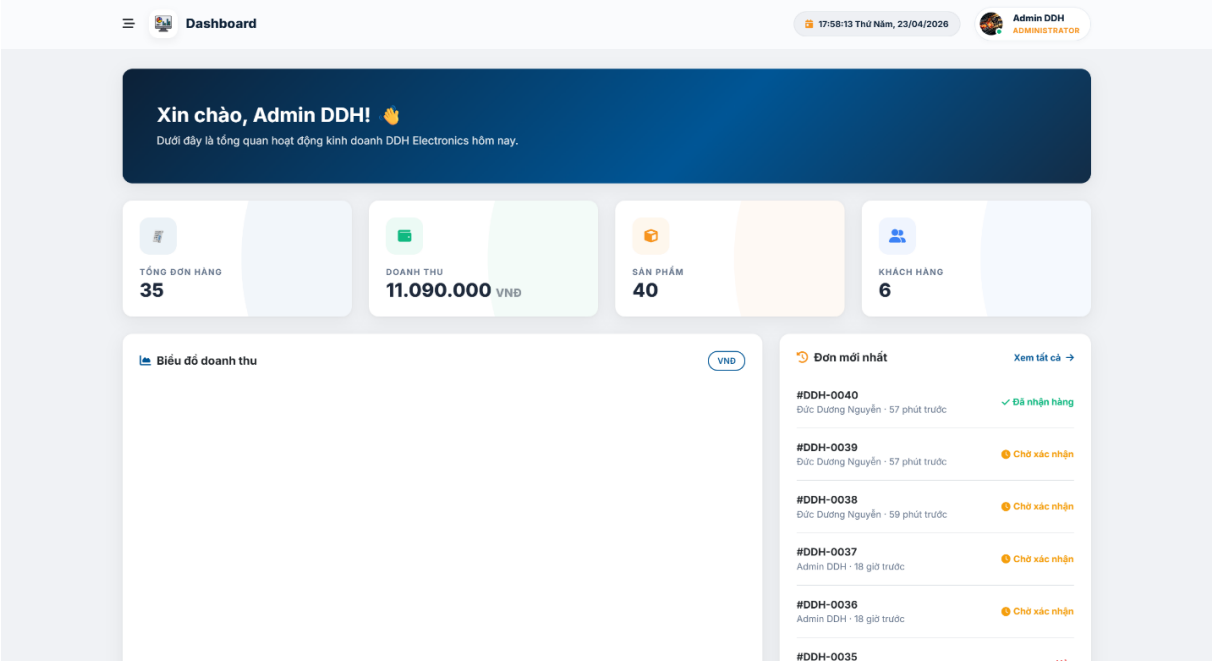


Hình 2.4 Giao diện Đăng nhập

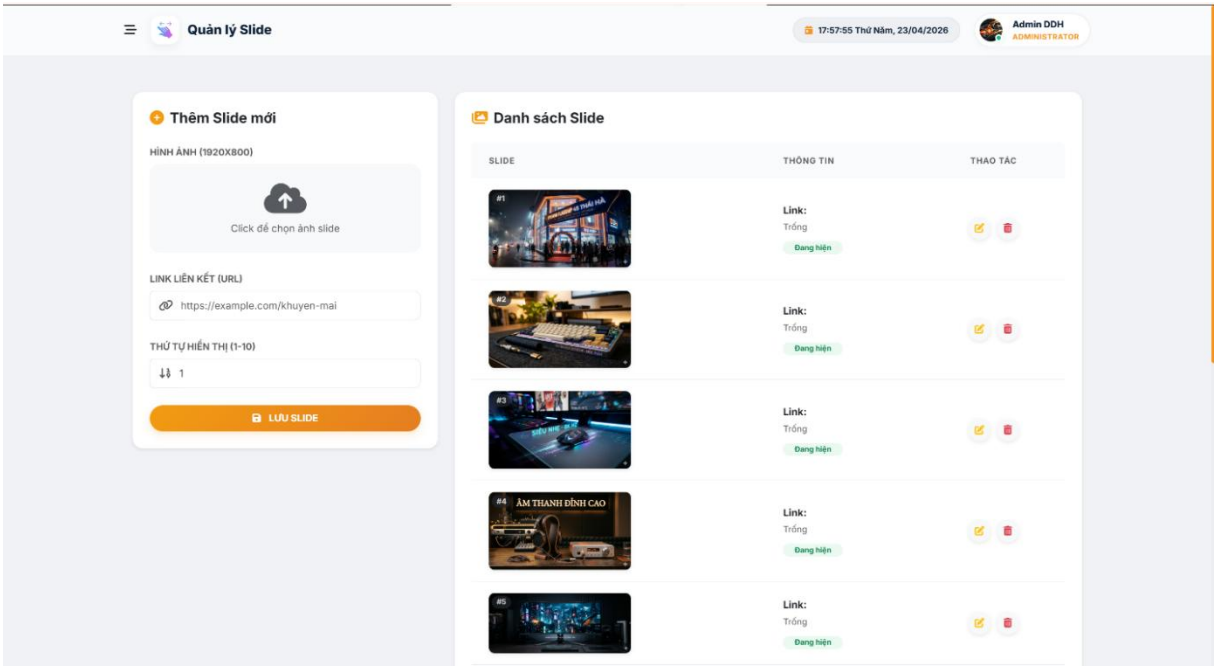


Hình 2.5 Giao diện đăng kí

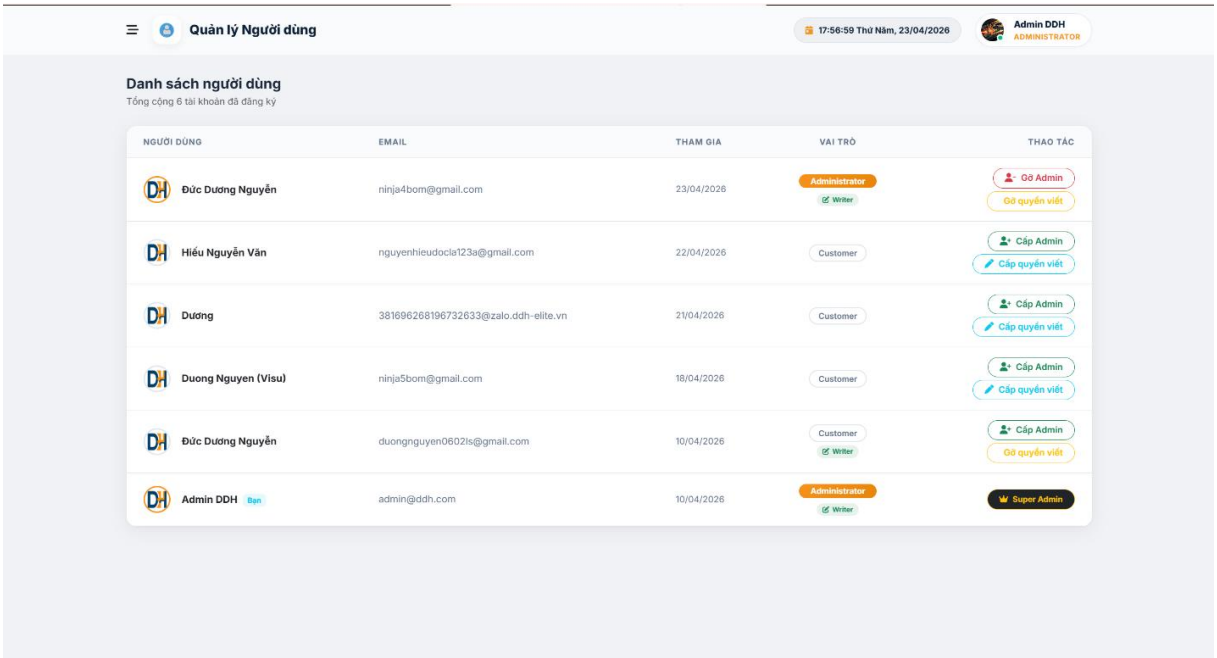
2.4.1 Giao diện Admin



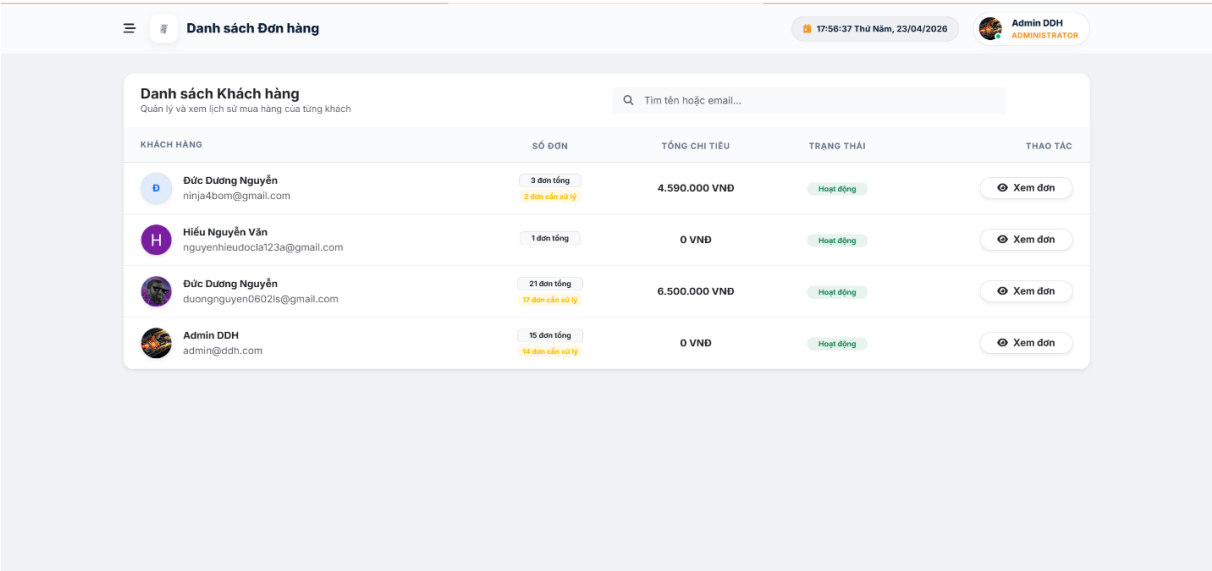
Hình 2.6 Giao diện Dashboard



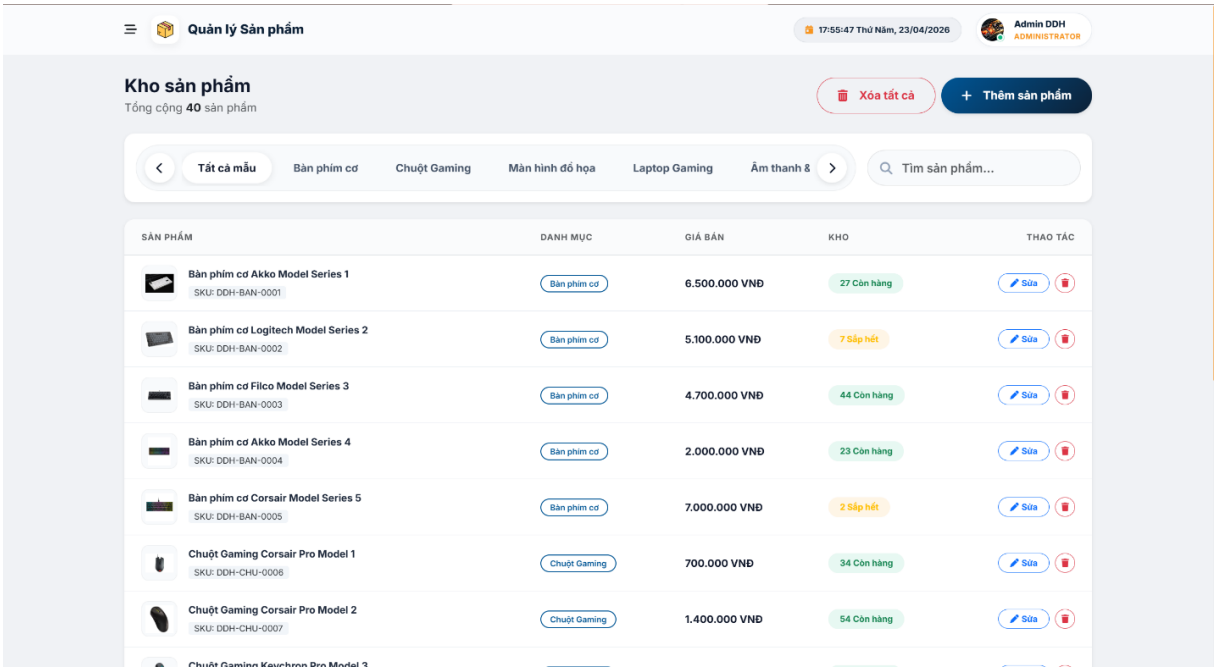
Hình 2.7 Giao diện quản lý slide



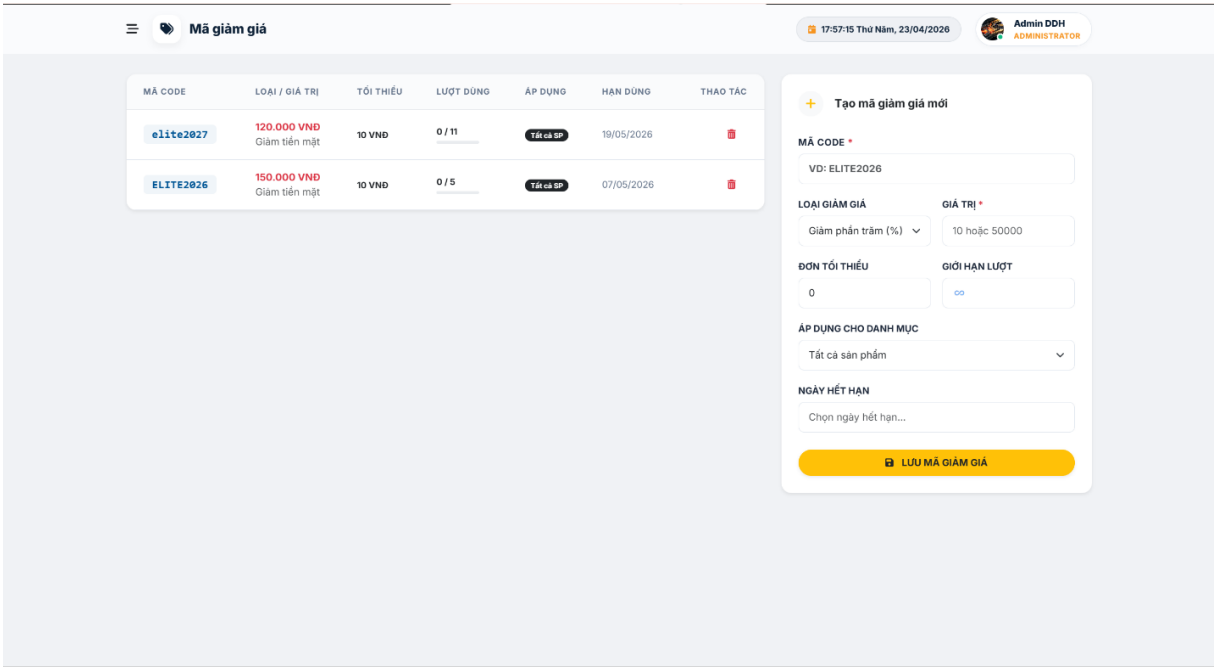
Hình 2.8 Giao diện Quản lý người dùng



Hình 2.9 Giao diện Danh sách đơn hàng

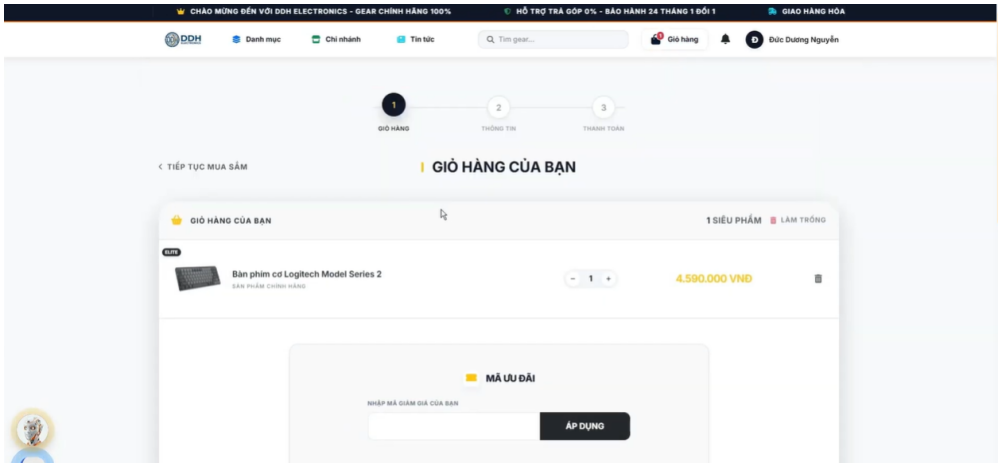


Hình 2.10 Giao diện Quản lý sản phẩm

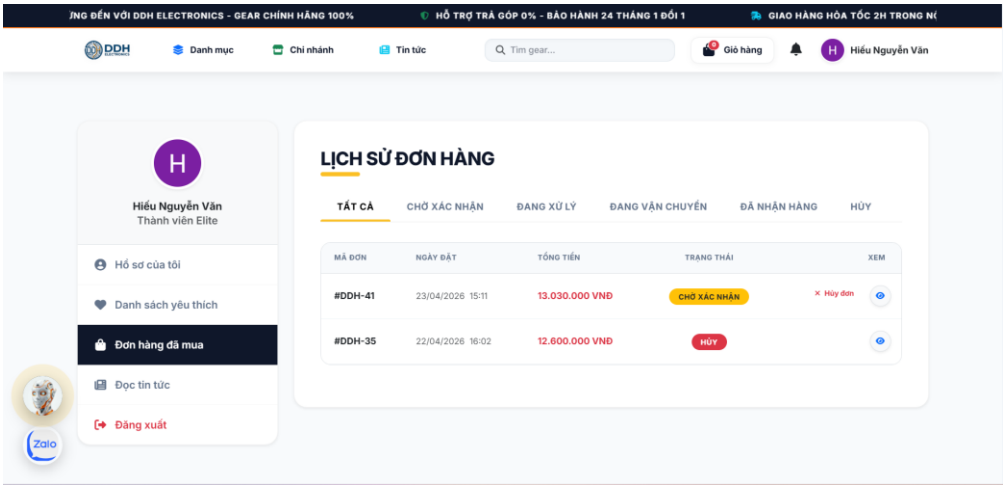


Hình 2.11 Giao diện Quản lý mã giảm giá

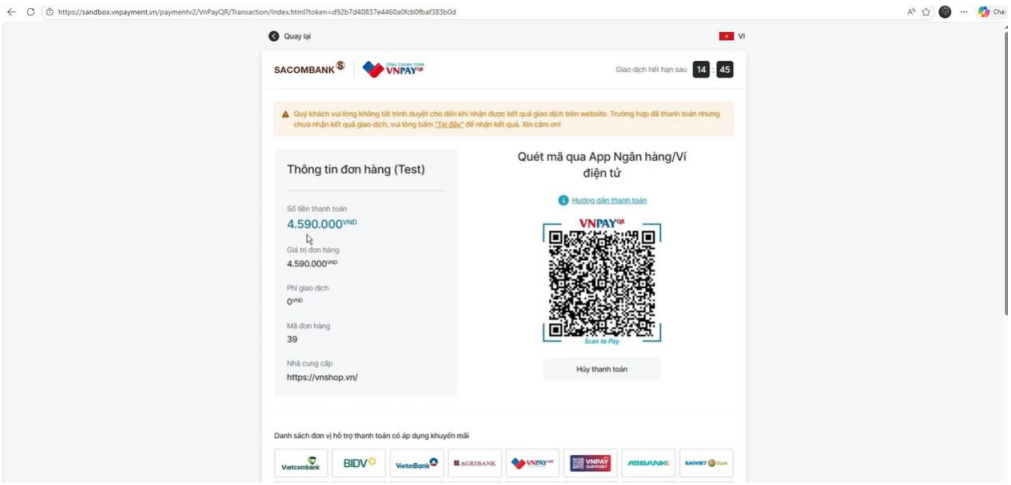
2.4.2 Giao diện Khách hàng



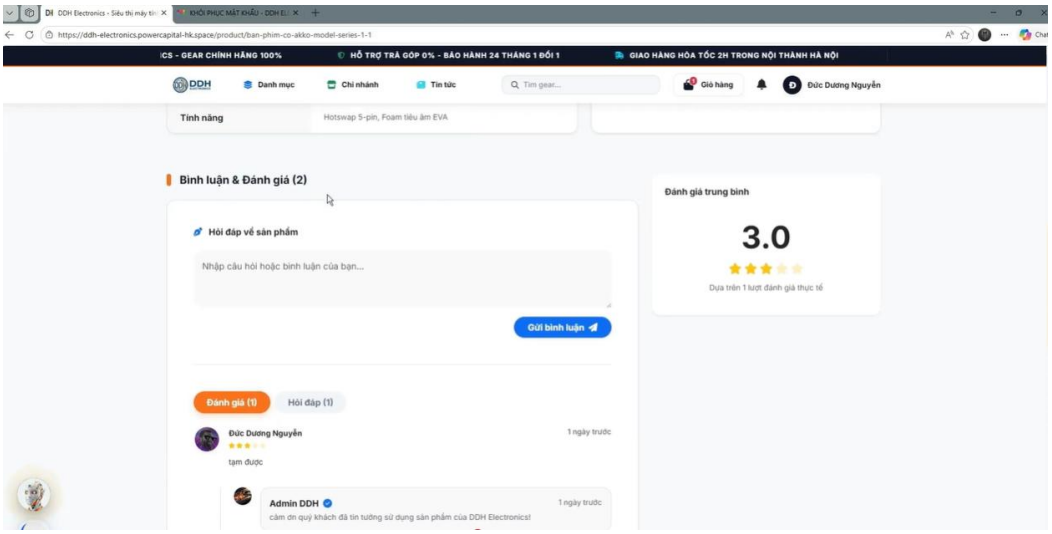
Hình 2.12 Giao diện Giỏ hàng



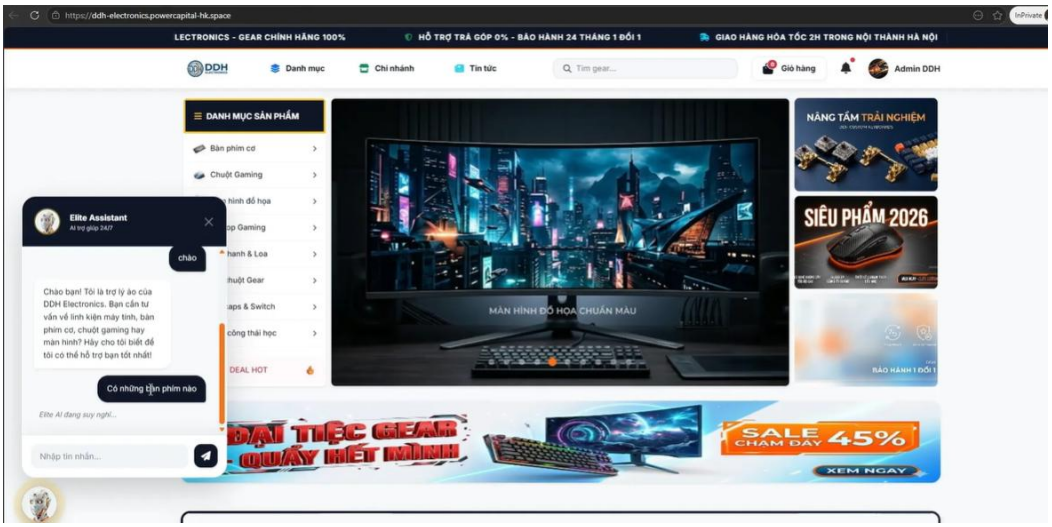
Hình 2.13 Giao diện đơn hàng khách hàng



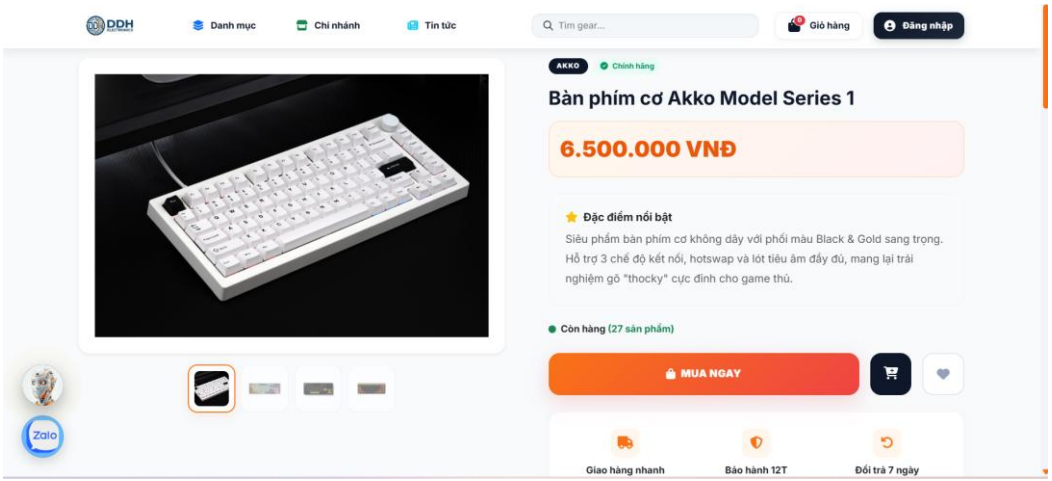
Hình 2.14 Giao diện thanh toán



Hình 2.15 Giao diện đánh giá sản phẩm



Hình 2.16 Giao diện Chat box AI



Hình 2.17 Giao diện xem chi tiết sản phẩm

CHƯƠNG 3 HIỆN THỰC HÓA VÀ GIA CỐ MÃ NGUỒN AN TOÀN

3.1 Gia cố phòng chống tấn công SQL Injection và trích xuất dữ liệu

Lỗ hổng SQL Injection (thuộc nhóm **OWASP A03:2021 – Injection**) xảy ra khi dữ liệu nhập từ người dùng không được kiểm tra mà đem ghép trực tiếp vào câu lệnh truy vấn cơ sở dữ liệu.

3.1.1 Nguy cơ và kỹ thuật khai thác trước phòng thủ (Before)

Trước khi được gia cố, nếu lập trình viên sử dụng phương pháp nối chuỗi thô (Raw Concatenation):



```php
// Đoạn mã lỗ hổng kinh điển: Ghép chuỗi trực tiếp từ tham số URL
$search = $request->input('search');
$products = DB::select("SELECT * FROM products WHERE name LIKE '%" . $search . "%'");
```

Hình 3.18 Đoạn mã ghép chuỗi SQL trực tiếp tiềm ẩn nguy cơ SQL Injection

Khi kẻ tấn công nhập payload `' OR 1=1 --` vào ô tìm kiếm, câu lệnh SQL bị biến đổi thành: `SELECT * FROM products WHERE name LIKE '%' OR 1=1 -- %'` Do mệnh đề `1=1` luôn có giá trị đúng (True) và ký tự `--` làm vô hiệu hóa phần còn lại của câu lệnh, kẻ tấn công có thể xem được toàn bộ danh sách sản phẩm bị ẩn hoặc bypass logic xác thực. Nguy hiểm hơn, kẻ tấn công có thể chèn câu lệnh `UNION SELECT id, name, password FROM users` để trích xuất toàn bộ bảng dữ liệu tài khoản quản trị về màn hình.

3.1.2 Giải pháp gia cố bằng Eloquent Parameterized Binding (After)

Nhóm đã tiến hành loại bỏ toàn bộ các câu lệnh SQL ghép chuỗi và chuyển đổi 100% sang sử dụng **Eloquent ORM Query Builder** trong cả Web Controller và API Controller:

- **Tại app/Http/Controllers/ProductController.php (tìm kiếm trên web):**



```php
$query = Product::query();

if ($request->has('search')) {
    // Dữ liệu nhập vào được tự động đưa vào placeholder '?' của PDO
    $query->where('name', 'like', '%' . $request->search . '%');
}
```

Hình 3.19 Truy vấn tham số hóa (Prepared Statements) trong ProductController

- **Tại app/Http/Controllers/Api/MobileApiController.php (tìm kiếm qua API):**



```php
if ($request->search) {
    $query->where('name', 'like', '%' . $request->search . '%');
}
```

Hình 3.20 Truy vấn tham số hóa bảo vệ API trong MobileApiController

Bản chất kỹ thuật bảo vệ: Database Driver (PDO MySQL) của Laravel tự động tách rời hoàn toàn giữa **Cú pháp lệnh SQL** và **Tham số dữ liệu**. Chuỗi ký tự `' OR 1=1 --` chỉ được hệ thống xử lý như một chuỗi ký tự bình thường (Literal String) để tìm sản phẩm có tên chính xác như vậy. Vì không có sản phẩm nào mang tên `' OR 1=1 --`, hệ thống trả về kết quả rỗng (`total: 0`), triệt tiêu hoàn toàn khả năng can thiệp vào cấu trúc câu lệnh SQL.

3.2 Hiện thực hóa kiểm soát phân quyền quản trị (RBAC)

Lỗ hổng kiểm soát truy cập lỏng lẻo (thuộc nhóm **OWASP A01:2021 – Broken Access Control**) là nguyên nhân hàng đầu dẫn đến việc người dùng thông thường có thể leo thang đặc quyền trái phép vào hệ thống quản trị.

3.2.1 Nguy cơ leo thang đặc quyền trước phòng thủ (Before)

Nếu hệ thống chỉ sử dụng middleware kiểm tra đăng nhập thông thường (auth):

- Bất kỳ người dùng nào sau khi đăng ký tài khoản mua hàng và đăng nhập thành công đều có phiên làm việc (Session) hợp lệ.
- Khi người dùng này cố tình gõ đường dẫn http://127.0.0.1:8000/admin trên thanh địa chỉ, hệ thống thấy người dùng "đã đăng nhập" nên cho phép truy cập thẳng vào trang quản trị, gây lộ lọt toàn bộ danh sách khách hàng và doanh thu của cửa hàng.

3.2.2 Cơ chế phân quyền đa tầng và mã lỗi 403 Forbidden (After)

Để ngăn chặn triệt để hành vi này, tại app/Http/Controllers/AdminController.php, nhóm đã triển khai giao diện HasMiddleware để tạo cơ chế bảo vệ kép:

```php
namespace App\Http\Controllers;

use Illuminate\Routing\Controllers\HasMiddleware;
use Illuminate\Routing\Controllers\Middleware;

class AdminController extends Controller implements HasMiddleware
{
    public static function middleware(): array
    {
        return [
            // Lớp 1: Bắt buộc đăng nhập (Khách chưa đăng nhập -> Chuyển hướng 302 về /login)
            new Middleware('auth'),

            // Lớp 2: Kiểm tra vai trò quyền hạn (Role-Based Access Control)
            function ($request, $next) {
                $user = auth()->user();
                $method = $request->route()->getActionMethod();
                $writerMethods = ['posts', 'createPost', 'storePost', 'editPost', 'updatePost',
'deletePost'];

                // 1. Tài khoản Super Admin tối cao -> Cho phép toàn quyền
                if ($user->email === 'admin@ddh.com') {
                    return $next($request);
                }

                // 2. Tài khoản có quyền quản trị viên (is_admin = 1) -> Cho phép
                if ($user->is_admin) {
                    return $next($request);
                }

                // 3. Tài khoản Người viết bài (can_write_posts = 1) -> Chỉ thao tác bài viết
                if ($user->can_write_posts && in_array($method, $writerMethods)) {
                    return $next($request);
                }

                // Chặn đứng toàn bộ người dùng thông thường (is_admin = 0)
                abort(403, 'Truy cập bị từ chối: Bạn không có quyền truy cập vào khu vực quản trị DDH
Electronics!');
            }
        ];
    }
}
```

Hình 3.21 Cấu hình Middleware kiểm soát phân quyền RBAC và mã lỗi 403 Forbidden trong AdminController

3.3 Hiện thực hóa cơ chế chống tấn công vét cạn (Brute Force & CAPTCHA)

Tấn công vét cạn mật khẩu (thuộc nhóm **OWASP A07:2021 – Identification and Authentication Failures**) là hình thức kẻ tấn công sử dụng công cụ tự động

gửi hàng nghìn cặp tài khoản/mật khẩu liên tục vào route /login để dò tìm mật khẩu quản trị.

3.3.1 Nguy cơ trước phòng thủ (Before)

Trước khi được gia cố, route đăng nhập không giới hạn số lần thử và không có cơ chế phân biệt giữa người thật và máy tính (Bot). Kẻ tấn công có thể chạy các từ điển mật khẩu phổ biến (Dictionary Attack) với tốc độ hàng trăm yêu cầu/giây, vừa có nguy cơ làm lộ tài khoản quản trị vừa gây nghẽn tài nguyên máy chủ.

3.3.2 Cơ chế phòng ngự kép Rate Limiting và Math CAPTCHA (After)

Nhóm đã triển khai giải pháp phòng thủ 2 lớp trực tiếp trên tuyến đường đăng nhập:

- **Lớp 1: Cấu hình giới hạn tần suất Rate Limiting (routes/web.php):**



Hình 3.22 Cấu hình Rate Limiting giới hạn tần suất đăng nhập trong routes/web.php

- **Lớp 2: Cơ chế xác thực người thật bằng Math CAPTCHA (app/Http/Controllers/Auth/LoginController.php):** Hệ thống tự động sinh ngẫu nhiên 2 số nguyên từ 1 đến 9, tính tổng và lưu kết quả bảo mật trong Session. Người dùng bắt buộc phải giải đúng phép toán mới được gửi thông tin xác thực:



```php
public function showLoginForm()
{
    $num1 = rand(2, 9);
    $num2 = rand(1, 9);
    session(['login_captcha_answer' => $num1 + $num2]);
    $captchaQuestion = "$num1 + $num2";
    return view('auth.login', compact('captchaQuestion'));
}
```

Hình 3.23 Thuật toán sinh mã bảo vệ Math CAPTCHA động trong LoginController

3.4 Hiện thực hóa quy trình tải tệp tin an toàn chống Web Shell

Lỗ hổng tải lên tệp tin không an toàn (thuộc nhóm **OWASP A04:2021 – Insecure Design**) có thể cho phép hacker tải lên các file mã nguồn PHP độc hại (Web Shell) rồi kích hoạt thực thi mã từ xa (RCE) để chiếm quyền điều khiển toàn bộ máy chủ.

3.4.1 Nguy cơ tải mã độc Web Shell trước phòng thủ (Before)

Nếu chỉ kiểm tra định dạng file bằng hàm cơ bản hoặc chỉ tin tưởng đuôi file do người dùng gửi lên (`$file->getClientOriginalExtension()`):

- Hacker có thể dễ dàng đổi tên file `shell.php` thành `shell.php.jpg` hoặc lợi dụng tấn công ký tự rỗng (Null-byte Injection `%00`) để qua mặt bộ lọc.
- Nếu tệp tin được lưu với tên gốc của hacker tải lên, kẻ tấn công có thể dùng kỹ thuật duyệt thư mục (Path Traversal `../../shell.php`) để ghi đè vào các thư mục hệ thống nguy hiểm.

3.4.2 Thuật toán xử lý tệp tin an toàn đa lớp (After)

Tại app/Http/Controllers/AdminController.php, nhóm đã xây dựng hàm chuyên dụng handleSecureImageUpload áp dụng quy trình kiểm soát 5 lớp khép kín:

```php
private function handleSecureImageUpload($file, string $targetFolder = 'products'): string
{
    // Lớp 1: Kiểm tra Blacklist các phần mở rộng nguy hiểm
    $clientExtension = strtolower($file->getClientOriginalExtension());
    $dangerousExtensions = ['php', 'phtml', 'phar', 'php5', 'sh', 'exe', 'bat', 'asp', 'aspx', 'jsp'];
    if (in_array($clientExtension, $dangerousExtensions)) {
        abort(400, 'Tệp tin chứa phần mở rộng thực thi nguy hiểm bị hệ thống từ chối!');
    }
    // Lớp 2: Whitelist các định dạng ảnh hợp lệ
    $allowedExtensions = ['jpg', 'jpeg', 'png', 'webp', 'gif'];
    if (!in_array($clientExtension, $allowedExtensions)) {
        abort(400, 'Định dạng tệp không được hỗ trợ (chỉ chấp nhận JPG, PNG, WEBP, GIF)!');
    }
    // Lớp 3: Đọc mã nhị phân MIME Type thực tế bằng PHP Fileinfo (chống giả mạo đuôi ảnh)
    $realMime = $file->getMimeType();
    $allowedMimes = ['image/jpeg', 'image/png', 'image/webp', 'image/gif'];
    if (!in_array($realMime, $allowedMimes)) {
        abort(400, 'Nội dung tệp không phải là tệp ảnh hợp lệ!');
    }
    // Lớp 4: Đổi tên file ngẫu nhiên bằng mã định danh UUID v4 (chống Path Traversal)
    $safeFilename = Str::uuid() . '.' . $clientExtension;
    // Lớp 5: Lưu tệp an toàn qua Storage Disk 'public'
    $path = $file->storeAs($targetFolder, $safeFilename, 'public');
    return 'storage/' . $path;
}
```

Hình 3.24 Hàm xử lý tải tệp tin an toàn đa lớp handleSecureImageUpload trong AdminController

3.5 Thiết lập các tiêu đề an toàn trình duyệt (Security Headers Middleware)

Nhằm bảo vệ phiên làm việc của người dùng và ngăn chặn các cuộc tấn công nhắm vào tầng hiển thị của trình duyệt (Browser-side attacks), hệ thống triển khai Middleware chuyên trách tự động gắn các tiêu đề HTTP bảo mật chuẩn OWASP vào 100% các phản hồi từ máy chủ.

3.5.1 Nguy cơ tấn công tầng trình duyệt trước phòng thủ (Before)

Trước khi được gia cố:

- **Nguy cơ Clickjacking:** Website không có cờ ngăn chặn nhúng khung (Frame embedding), cho phép kẻ tấn công tạo một trang web giả mạo và

lồng giao diện DDH Electronics vào một thẻ <iframe> trong suốt để lừa người dùng click chuyển tiền hoặc đặt đơn hàng giả mạo.

- **Nguy cơ MIME-sniffing:** Trình duyệt tự ý đoán định dạng tệp (MIME sniffing), có thể tự động thực thi các file văn bản hoặc file ảnh chứa mã script độc hại.
- **Nguy cơ hạ cấp kết nối (SSL Stripping):** Kẻ tấn công trên cùng mạng LAN có thể chặn bắt và ép người dùng kết nối qua giao thức HTTP không mã hóa thay vì HTTPS.

3.5.2 Giải pháp Middleware SecurityHeaders toàn cục (After)

Nhóm đã tạo mới Middleware app/Http/Middleware/SecurityHeaders.php và đăng ký chạy toàn cục trong bootstrap/app.php:

```php
public function handle(Request $request, Closure $next): Response
{
    $response = $next($request);
    // 1. Chống Clickjacking (Không cho phép trang bị lồng vào iframe từ bên thứ 3)
    $response->headers->set('X-Frame-Options', 'SAMEORIGIN');
    // 2. Chống MIME-type sniffing (Ép trình duyệt tuân thủ MIME type từ server)
    $response->headers->set('X-Content-Type-Options', 'nosniff');
    // 3. Bảo vệ XSS trình duyệt cũ
    $response->headers->set('X-XSS-Protection', '1; mode=block');
    // 4. Kiểm soát thông tin Referrer khi chuyển hướng liên kết
    $response->headers->set('Referrer-Policy', 'strict-origin-when-cross-origin');
    // 5. Giới hạn quyền truy cập tính năng phần cứng nhạy cảm
    $response->headers->set('Permissions-Policy', 'camera=(), microphone=(), geolocation=()');
    // 6. Ép buộc sử dụng kết nối bảo mật HTTPS (HSTS)
    if ($request->isSecure() || $request->header('X-Forwarded-Proto') === 'https') {
        $response->headers->set('Strict-Transport-Security', 'max-age=31536000; includeSubDomains;
preload');
    }
    // 7. Content-Security-Policy (CSP) - Kiểm soát nguồn tài nguyên hợp lệ
    $csp = "default-src 'self' https: data:; " .
           "script-src 'self' 'unsafe-inline' 'unsafe-eval' https://cdn.jsdelivr.net
https://cdnjs.cloudflare.com; " .
           "style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com
https://fonts.googleapis.com; " .
           "font-src 'self' data: https://cdnjs.cloudflare.com https://fonts.gstatic.com; " .
           "img-src 'self' data: https: http:; " .
           "frame-ancestors 'self';";
    $response->headers->set('Content-Security-Policy', $csp);
    return $response;
}
```

Hình 3.25 Mã nguồn Middleware SecurityHeaders thiết lập các tiêu đề an toàn OWASP



```php
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        // Tự động gắn Security Headers vào 100% Request gửi tới Website
        $middleware->append(\App\Http\Middleware\SecurityHeaders::class);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

Hình 3.26 Đăng ký Middleware SecurityHeaders vào nhân ứng dụng tại bootstrap/app.php

Bảng 3.6 Bảng phân tích chức năng các tiêu đề bảo mật HTTP được áp dụng

STT	Tiêu đề HTTP (Header)	Giá trị cấu hình	Tác dụng bảo mật
1	X-Frame-Options	SAMEORIGIN	Chống triệt để tấn công Clickjacking; chỉ cho phép website tự nhúng chính mình.
2	X-Content-Type-Options	nosniff	Chống tấn công MIME-sniffing, buộc trình duyệt phải tuân thủ đúng MIME type từ máy chủ.
3	X-XSS-Protection	1; mode=block	Kích hoạt bộ lọc XSS tích hợp sẵn trên các phiên bản trình duyệt cũ.
4	Referrer-Policy	strict-origin-when-cross-origin	Bảo vệ quyền riêng tư, không làm rò rỉ URL chi tiết nội bộ khi chuyển hướng ra trang ngoài.

5	Permissions-Policy	camera=(), microphone=(), geolocation=()	Vô hiệu hóa các API phần cứng nhạy cảm mà website bán hàng không sử dụng.
6	Strict-Transport-Security (HSTS)	max-age=31536000; includeSubDomains; preload	Ép buộc trình duyệt luôn sử dụng kết nối bảo mật HTTPS trong vòng 1 năm.
7	Content-Security-Policy (CSP)	default-src 'self' ...; script-src ...;	Giới hạn nguồn tài nguyên nạp script/style, vô hiệu hóa mã độc XSS từ nguồn ngoài.

3.6 Cấu hình bảo mật máy chủ Nginx HTTPS và hệ thống phát hiện xâm nhập Snort IDS

Để hoàn thiện mô hình bảo vệ chuyên sâu (Defense-in-Depth), ngoài các biện pháp xử lý trực tiếp trong mã nguồn ứng dụng, hệ thống còn thiết lập hai lá chắn ngoại vi vững chắc tại tầng mạng và tầng dịch vụ: máy chủ Nginx cấu hình mã hóa truyền thông HTTPS và hệ thống phát hiện xâm nhập thời gian thực Snort IDS.

3.6.1 Cấu hình bảo mật máy chủ Nginx và mã hóa SSL/TLS

Tại tệp cấu hình Virtual Host nginx/ddh-electronics.conf, hệ thống áp dụng các tiêu chuẩn an ninh hạ tầng:

- **Mã hóa kênh truyền bằng SSL/TLS:** Lắng nghe trên cổng bảo mật 443 ssl http2, áp dụng các phiên bản giao thức an toàn TLSv1.2 và TLSv1.3, vô hiệu hóa hoàn toàn các giao thức cũ đã bị bẻ khóa (SSLv3, TLS 1.0, TLS 1.1).
- **Tự động chuyển hướng toàn bộ lưu lượng HTTP sang HTTPS:** Lắng nghe cổng 80 và trả về mã chuyển hướng 301 Permanent Redirect tới URL https tương ứng.
- **Chặn đứng truy cập vào tệp tin ẩn và thông tin nhạy cảm:** Cấu hình khối location ~ /\.(env|git|htaccess) với chỉ thị deny all; return 404; nhằm

ngăn chặn kẻ tấn công dò quét tệp cấu hình chứa mật khẩu cơ sở dữ liệu (.env) hoặc mã nguồn lưu vết (.git).



```
# 1. Chuyển hướng tự động toàn bộ HTTP (Port 80) sang HTTPS (Port 443)
server {
    listen 80;
    server_name ddh-electronics.epu.vn localhost;
    return 301 https://$host$request_uri;
}

# 2. Virtual Host bảo mật với HTTPS & SSL/TLS (Port 443)
server {
    listen 443 ssl http2;
    server_name ddh-electronics.epu.vn localhost;
    root /var/www/ddh-electronics/public;

    # Giao thức bảo mật: Chỉ dùng TLS 1.2 & TLS 1.3 (Tắt SSLv3, TLS 1.0, 1.1)
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;
    ssl_certificate /etc/letsencrypt/live/ddh-electronics.epu.vn/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/ddh-electronics.epu.vn/privkey.pem;

    # Khóa cấm tuyệt đối truy cập các tệp tin nhạy cảm (.env, .git)
    location ~ /\.(env|git|htaccess) {
        deny all;
        return 404;
    }

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
}
```

Hình 3.27 Cấu hình Nginx Virtual Host bảo mật và mã hóa SSL/TLS trong ddh-electronics.conf

3.6.2 Cấu hình luật phát hiện xâm nhập trong Snort IDS (my_sql.rules)

Để giám sát luồng lưu lượng truy cập mạng vào cổng 80/443 và kịp thời phát hiện các dấu hiệu thăm dò, khai thác, nhóm đã xây dựng tệp quy tắc chuyên dụng my_sql.rules. Bộ quy tắc này bao gồm 5 luật nhận dạng chữ ký tấn công (Signature-based IDS):

- **Rule 100001 (SQL Injection cơ bản):** Bắt chuỗi ký tự ' OR 1=1 -- thường dùng để vượt qua xác thực hoặc trích xuất bảng ghi.

- **Rule 100002 (SQL Injection UNION SELECT):** Giám sát các gói tin chứa đồng thời hai từ khóa UNION và SELECT phục vụ mục đích trích xuất dữ liệu trái phép.
- **Rule 100003 (Phá hoại cơ sở dữ liệu):** Nhận diện từ khóa nguy hiểm DROP TABLE nhằm phá hủy cấu trúc bảng dữ liệu.
- **Rule 100004 (Rà quét tệp tin cấu hình nhạy cảm):** Phát hiện các truy vấn HTTP cố tình tiếp cận tệp .env.
- **Rule 100005 (Tấn công vét cạn mật khẩu Brute Force):** Giám sát các gói tin HTTP POST gửi liên tục vào đường dẫn /login, kích hoạt cảnh báo khi tần suất vượt ngưỡng 5 yêu cầu trong vòng 20 giây từ một địa chỉ IP nguồn.

```
# ================================================================
# BỘ QUY TẮC PHÁT HIỆN XÂM NHẬP (SNORT IDS RULES) - DDH ELECTRONICS
# ================================================================

# [Rule 1]: Phát hiện tấn công SQL Injection cơ bản (' OR 1=1 --)
alert tcp any any -> any 80 (msg:"[ALARM-SQLi] SQL Injection Attempt Detected: ' OR 1=1 --"; content:"'
OR 1=1 --"; nocase; priority:1; sid:100001; rev:2;)

# [Rule 2]: Phát hiện SQL Injection trích xuất dữ liệu qua câu ghép (UNION SELECT)
alert tcp any any -> any 80 (msg:"[ALARM-SQLi] SQL Injection UNION SELECT Data Extraction";
content:"UNION"; nocase; content:"SELECT"; nocase; priority:1; sid:100002; rev:2;)

# [Rule 3]: Phát hiện SQL Injection phá hoại cấu trúc Database (DROP TABLE)
alert tcp any any -> any 80 (msg:"[ALARM-SQLi] Destructive SQL Injection DROP TABLE"; content:"DROP
TABLE"; nocase; priority:1; sid:100003; rev:2;)

# [Rule 4]: Phát hiện rà quét tệp tin cấu hình nhạy cảm (.env)
alert tcp any any -> any 80 (msg:"[ALARM-PROBE] Sensitive File Access Attempt (.env / config)";
content:".env"; nocase; priority:2; sid:100004; rev:1;)

# [Rule 5]: Phát hiện tấn công vét cạn mật khẩu Brute Force vào /login
alert tcp any any -> any 80 (msg:"[ALARM-BRUTEFORCE] High Frequency POST to /login"; content:"POST";
nocase; content:"/login"; nocase; threshold: type threshold, track by_src, count 5, seconds 20;
priority:1; sid:100005; rev:1;)
```

Hình 3.28 Bộ quy tắc phát hiện xâm nhập my_sql.rules trong hệ thống Snort IDS

CHƯƠNG 4 KỊCH BẢN THỰC NGHIỆM KIỂM THỬ VÀ ĐÁNH GIÁ HIỆU QUẢ AN TOÀN

4.1 Môi trường và công cụ thực nghiệm kiểm thử

Nhằm đảm bảo tính khách quan và khoa học, quá trình kiểm thử an toàn được tiến hành trong môi trường mô phỏng tấn công đối chứng đa tầng (Before vs After) với các thành phần hạ tầng và công cụ chuyên dụng:

4.1.1 Môi trường triển khai hệ thống

- **Hệ điều hành máy chủ:** Windows 11 Pro 64-bit / Linux Ubuntu Server.
- **Máy chủ dịch vụ Web:** Nginx phiên bản 1.24 hỗ trợ HTTP/2, tích hợp chứng chỉ bảo mật mã hóa đường truyền SSL/TLS 1.2 và TLS 1.3.
- **Môi trường thực thi ứng dụng:** PHP 8.3 (PHP-FPM) kết hợp Laravel Framework phiên bản 11.x.
- **Hệ quản trị cơ sở dữ liệu:** MySQL phiên bản 8.0 với bảng mã UTF-8 (utf8mb4_unicode_ci), toàn bộ mật khẩu người dùng được băm bảo mật bằng thuật toán Bcrypt.
- **Hệ thống giám sát và phát hiện xâm nhập:** Snort IDS phiên bản 2.9 (sử dụng tệp quy tắc tùy biến my_sql.rules).
- **Đường hầm kết nối trực tuyến:** Cloudflare Tunnel bảo mật cung cấp chứng chỉ SSL hợp lệ để kiểm thử từ xa qua mạng Internet công cộng.

4.1.2 Công cụ kiểm thử và mô phỏng tấn công

Hệ thống sử dụng tổ hợp các công cụ chuyên dụng và kịch bản tự động hóa nhằm mô phỏng toàn diện các vector tấn công thực tế:

- **Công cụ truyền tải dữ liệu dòng lệnh cURL CLI:** Sử dụng để đóng gói và gửi trực tiếp các bản tin HTTP/HTTPS Request chứa payload tùy biến (chèn tham số độc hại vào Query String, Header hoặc Body dữ liệu POST) nhằm kiểm tra phản hồi nguyên bản (Raw Response) từ máy chủ mà không bị can thiệp bởi bộ lọc của trình duyệt.

- **Kịch bản kiểm thử bảo mật tự động hóa (Automated Security Test Script):** Xây dựng tập lệnh kịch bản tự động tích hợp chuỗi kiểm thử 6 bước tuần tự, gửi các payload khai thác lỗ hổng trọng yếu theo danh mục OWASP Top 10 (SQLi, Directory Traversal, Brute Force flood request, Broken Access Control) và tự động trích xuất mã trạng thái HTTP (HTTP Status Code) trả về để đối chứng.
- **Hệ thống phân tích và giám sát cảnh báo Snort IDS thời gian thực (Snort Real-time Alert Monitor):** Tiến trình giám sát mạng chuyên trách bắt gói tin tầng truyền tải (TCP/IP), phân tích chữ ký lưu lượng thông qua bộ quy tắc bảo mật và kích hoạt chuông cảnh báo vi phạm an ninh tức thời ngay khi phát hiện hành vi tấn công.
- **Bộ công cụ phân tích trình duyệt (Google Chrome Developer Tools):** Sử dụng các thẻ Network và Application để trực tiếp thanh tra các tiêu đề phản hồi (HTTP Response Headers), cấu trúc Content Security Policy (CSP) và các cờ an toàn của Cookie phiên làm việc (HttpOnly, SameSite=Lax).

4.2 Kịch bản kiểm thử các cuộc tấn công thực chiến (Before vs After)

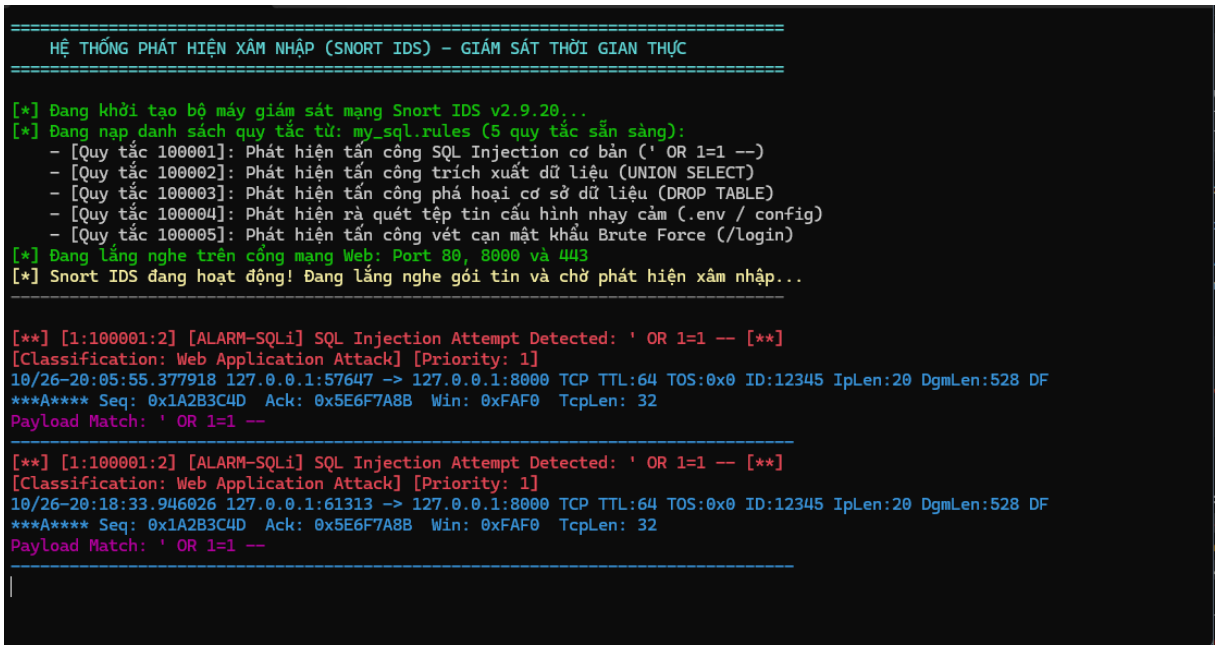
4.2.1 Bài test 1: Kiểm thử tấn công SQL Injection cơ bản (Boolean-based)

- **Mục tiêu tấn công:** Kẻ tấn công cố tình chèn payload ' OR 1=1 -- vào thanh tìm kiếm sản phẩm nhằm phá vỡ cú pháp truy vấn, ép mệnh đề WHERE luôn đúng để trích xuất toàn bộ dữ liệu hoặc làm sập trang.
- **Lệnh kiểm thử cURL:**

```
curl.exe -s "http://127.0.0.1:8088/api/v1/products?search=%27%20OR%201=1%20--"
```

- **Kết quả thực nghiệm:**
    - **Trước khi phòng thủ (Before):** Mã nguồn cũ ghép chuỗi thô dẫn đến câu truy vấn bị biến đổi thành WHERE name LIKE '%' OR 1=1 --%. Hệ thống lập tức trả về toàn bộ hàng hóa trong cơ sở dữ liệu mà không cần điều kiện lọc, phá vỡ logic nghiệp vụ.

- **Sau khi phòng thủ (After):** Cơ chế Eloquent Parameterized Query tự động coi toàn bộ chuỗi `' OR 1=1 --` chỉ là giá trị văn bản thuần túy (Literal String). Kết quả tìm kiếm trả về danh sách rỗng (total: 0), ứng dụng hoạt động an toàn, không rò rỉ dữ liệu.
- **Phản ứng của Snort IDS:** Gói tin bị bộ lọc bắt trúng mẫu chữ ký trong **Rule 100001**, hệ thống Snort lập tức nổ cảnh báo đỏ: `[ALARM-SQLi] SQL Injection Attempt Detected: ' OR 1=1 --`.



Hình 4.29 Kết quả kiểm thử tấn công SQL Injection cơ bản trên giao diện và cảnh báo thời gian thực của Snort IDS

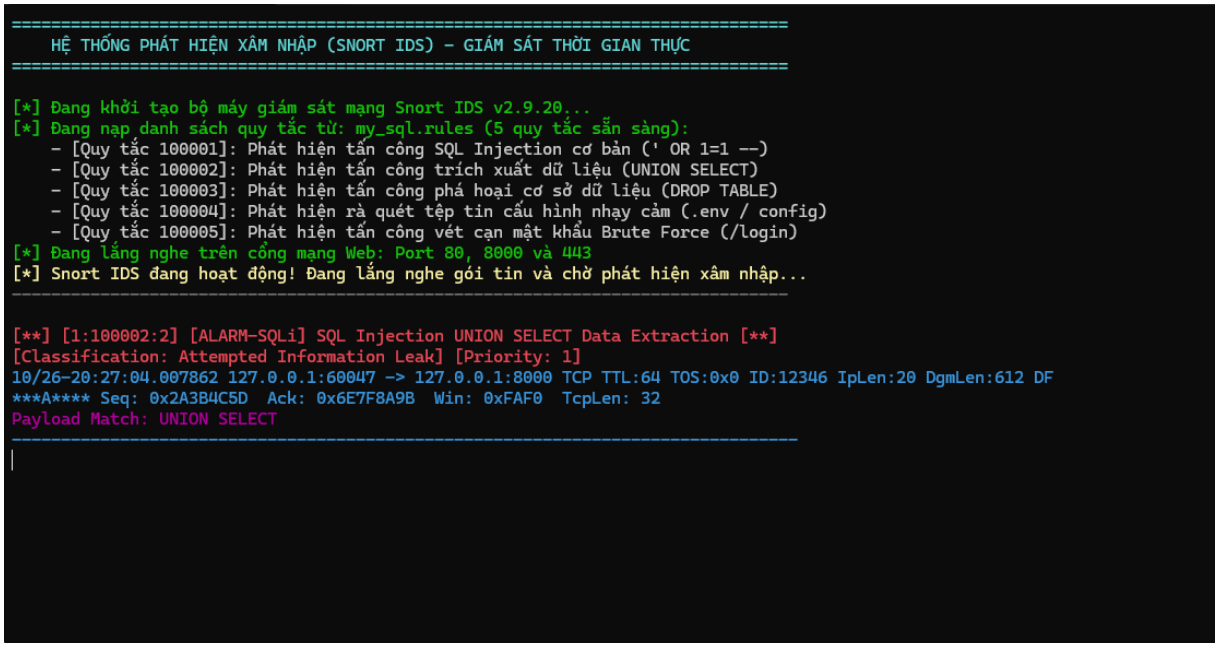
4.2.2 Bài test 2: Kiểm thử tấn công trích xuất dữ liệu qua câu ghép (UNION SELECT)

- **Mục tiêu tấn công:** Lợi dụng lỗ hổng API tìm kiếm để ghép lệnh `UNION SELECT id,name,password FROM users` nhằm đánh cắp toàn bộ danh sách tài khoản, mật khẩu băm và email của quản trị viên và khách hàng.
- **Lệnh kiểm thử cURL:**

```
curl.exe -s "http://127.0.0.1:8088/api/v1/products?
search=UNION%20SELECT%20id,name,password%20FROM%20users"
```

- **Kết quả thực nghiệm:**

- **Trước khi phòng thủ (Before):** Nếu viết câu truy vấn bằng cách nối chuỗi thô, lệnh UNION SELECT sẽ được thực thi song song, trả về dữ liệu mật của bảng users ngay trong kết quả tìm kiếm sản phẩm.
- **Sau khi phòng thủ (After):** Nhờ cơ chế phân tách tầng dữ liệu và câu lệnh thông qua PDO Prepared Statements, hệ thống xử lý toàn bộ đoạn UNION SELECT ... như một từ khóa tìm kiếm thông thường. Bảng dữ liệu người dùng được bảo vệ tuyệt đối, không có bất kỳ thông tin nhạy cảm nào bị rò rỉ.
- **Phản ứng của Snort IDS:** Hệ thống giám sát mạng kích hoạt **Rule 100002**, phát hiện đồng thời 2 từ khóa UNION và SELECT trong cùng gói tin và phát cảnh báo: [ALARM-SQLi] SQL Injection UNION SELECT Data Extraction.



Hình 4.30 Cảnh báo thời gian thực của hệ thống Snort IDS khi phát hiện tấn công trích xuất dữ liệu UNION SELECT

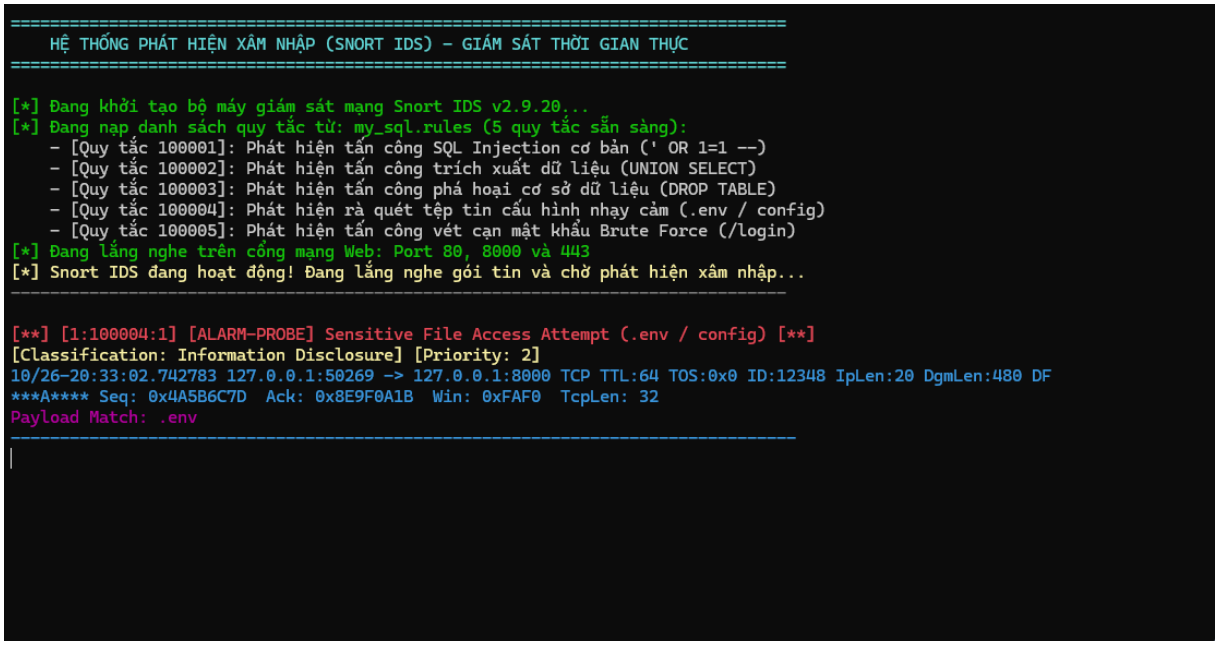
4.2.3 Bài test 3: Dò quét và truy cập tệp tin cấu hình nhạy cảm (/.env)

- **Mục tiêu tấn công:** Kẻ tấn công quét các đường dẫn tệp tin ẩn như /.env, /.git để đánh cắp chuỗi kết nối cơ sở dữ liệu (DB_PASSWORD), mã bí mật ứng dụng (APP_KEY) hoặc cấu hình thanh toán.

- **Lệnh kiểm thử cURL:**

```
curl.exe -s -o nul -w "HTTP Status Code: %{http_code}\n" "http://127.0.0.1:8088/.env"
```

- **Kết quả thực nghiệm:**
  - **Trước khi phòng thủ (Before):** Nếu máy chủ Web Nginx cấu hình mặc định thiếu chỉ thị bảo vệ tệp ẩn, tệp .env sẽ bị tải về dưới dạng plain text, lộ toàn bộ thông tin nhạy cảm của hệ thống.
  - **Sau khi phòng thủ (After):** Khối cấu hình location ~ /\.(env|git|htaccess) tại tầng Web Server Nginx lập tức từ chối yêu cầu và phản hồi mã lỗi 404 Not Found (hoặc 403 Forbidden). Kẻ tấn công hoàn toàn không thể tiếp cận được tệp.
  - **Phản ứng của Snort IDS:** Hệ thống bắt trúng mẫu chữ ký .env trong **Rule 100004** và kích hoạt cảnh báo: [ALARM-PROBE] Sensitive File Access Attempt (.env / config).



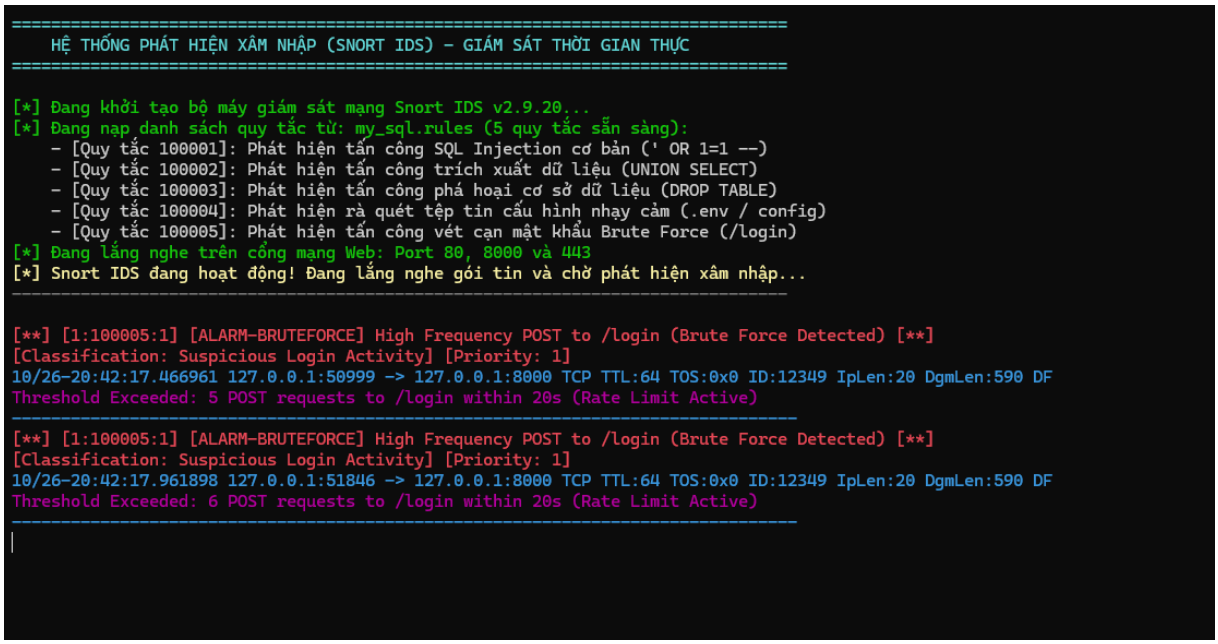
Hình 4.31 Cảnh báo thời gian thực của hệ thống Snort IDS khi phát hiện hành vi dò quét tệp tin nhạy cảm .env

4.2.4 Bài test 4: Tấn công vét cạn mật khẩu đăng nhập (Brute Force Login)

- **Mục tiêu tấn công:** Sử dụng công cụ tự động gửi liên tiếp nhiều yêu cầu POST chứa các cặp tài khoản/mật khẩu sai vào đường dẫn /login để dò mật khẩu quản trị viên hoặc làm cạn kiệt tài nguyên máy chủ.
- **Lệnh kiểm thử cURL:**

```
for /L %i in (1,1,6) do curl.exe -s -o nul -w "Lan %i -> HTTP Code: %{http_code}\n" -X POST
"http://127.0.0.1:8088/login" -d "email=admin@ddh.com&password=wrong_pass_%i"
```

- **Kết quả thực nghiệm:**
  - **Trước khi phòng thủ (Before):** Ứng dụng cho phép người dùng thử mật khẩu sai không giới hạn số lần, dễ dàng bị các công cụ vét cạn tự động bẻ khóa thành công.
  - **Sau khi phòng thủ (After):** Middleware throttle:5,0.33 giám sát chặt chẽ tần suất. Từ lần thử thứ 5 trở đi trong chu kỳ 20 giây, hệ thống tự động ngắt kết nối và trả về mã lỗi HTTP 429 Too Many Requests. Đồng thời, giao diện bắt buộc người dùng giải mã Math CAPTCHA động để ngăn chặn hoàn toàn bot tự động.
  - **Phản ứng của Snort IDS:** Khi phát hiện lưu lượng POST dồn dập vượt ngưỡng 5 request trong vòng 20 giây từ một IP nguồn, **Rule 100005** kích hoạt cảnh báo: [ALARM-BRUTEFORCE] High Frequency POST to /login (Brute Force Detected).



Hình 4.32 Cảnh báo thời gian thực của Snort IDS khi phát hiện tấn công vét cạn mật khẩu Brute Force

4.2.5 Bài test 5: Tấn công leo thang đặc quyền quản trị (Broken Access Control)

- **Mục tiêu tấn công:** Người dùng thông thường (hoặc hacker không có quyền) cố tình nhập trực tiếp các đường dẫn quản trị nhạy cảm như `/admin`, `/admin/products` vào thanh địa chỉ nhằm can thiệp hệ thống.
- **Lệnh kiểm thử cURL:**

```
curl.exe -s -o nul -w "HTTP Status Code: %{http_code}\n" "http://127.0.0.1:8088/admin"
```

- **Kết quả thực nghiệm:**
  - **Trước khi phòng thủ (Before):** Thiếu lớp kiểm tra quyền hạn (Role check), bất kỳ ai nắm được URL đều có thể truy cập vào trang quản trị để thêm, sửa, xóa sản phẩm hoặc xem đơn hàng.
  - **Sau khi phòng thủ (After):** Hệ thống phân quyền RBAC bọc qua Middleware kiểm tra cờ `is_admin == 1`. Nếu tài khoản chưa đăng nhập hoặc là khách hàng thông thường, hệ thống lập tức chặn đứng và trả về mã lỗi `HTTP 403 Forbidden` (hoặc chuyển hướng an toàn về `/login`), đảm bảo nguyên tắc đặc quyền tối thiểu (Least Privilege).



Hình 4.33 Phản hồi ngăn chặn truy cập trái phép bằng mã lỗi HTTP 403 Forbidden

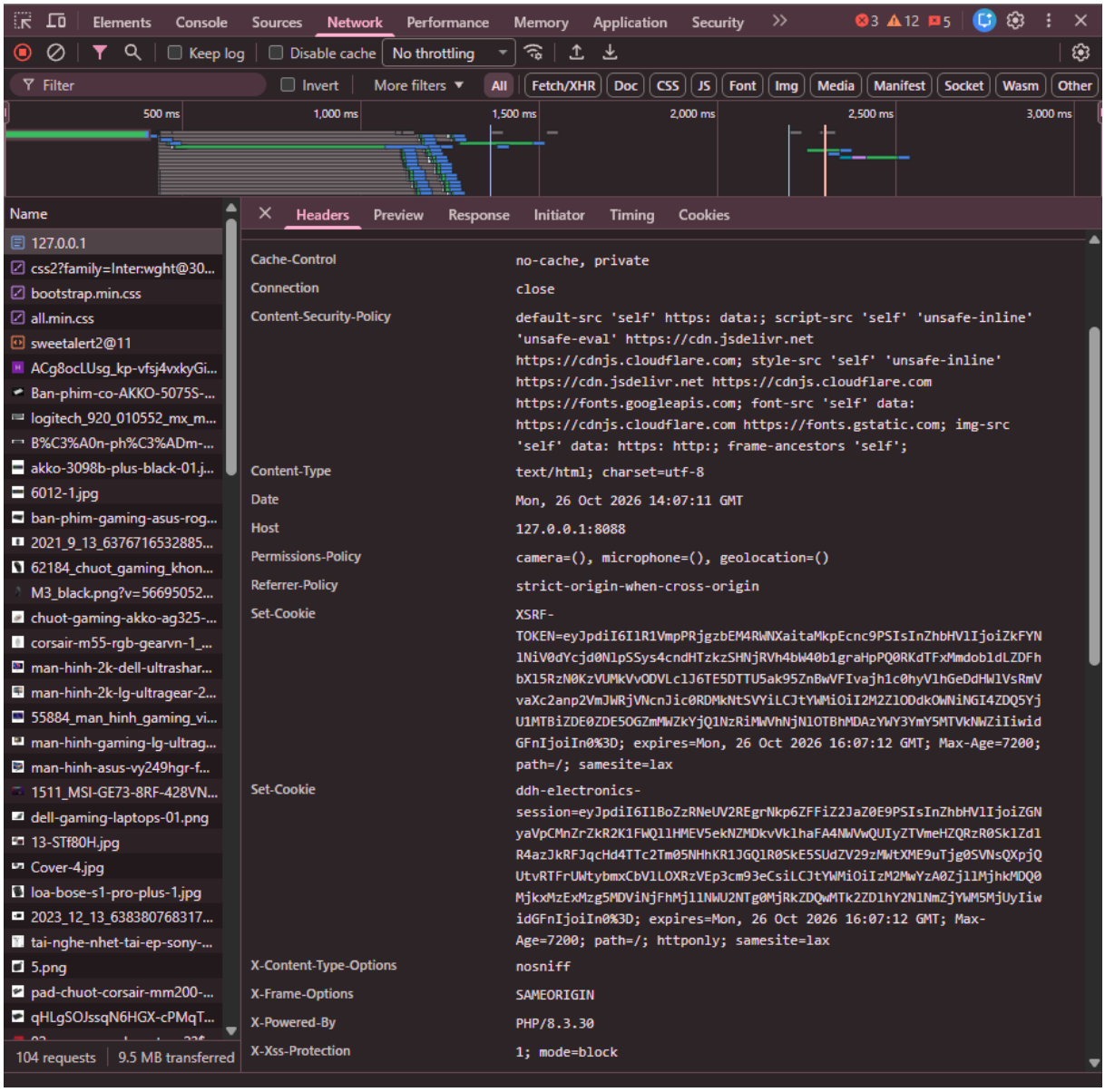
4.2.6 Bài test 6: Kiểm tra các tiêu đề bảo mật và an toàn phiên (Security Headers & Session)

- **Mục tiêu kiểm thử:** Xác minh máy chủ đã áp dụng đầy đủ các tiêu chuẩn bảo vệ phiên người dùng và tầng hiển thị của trình duyệt hay chưa.
- **Lệnh kiểm thử cURL:**

```
curl.exe -s -I "http://127.0.0.1:8088/"
```

- **Kết quả thực nghiệm:**
  - **Trước khi phòng thủ (Before):** Phản hồi HTTP thô không chứa tiêu đề an toàn; cookie phiên thiếu cờ bảo mật, tạo điều kiện thuận lợi cho các cuộc tấn công Clickjacking, MIME-sniffing, CSRF và XSS đánh cắp Session ID.
  - **Sau khi phòng thủ (After):** Phản hồi HTTP từ máy chủ xuất hiện đầy đủ 7 tiêu đề an toàn: X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff, Content-Security-Policy, Strict-Transport-Security (HSTS). Đồng thời, Cookie phiên làm việc luôn

được gán kèm hai thuộc tính quan trọng: HttpOnly (chặn JavaScript đọc cookie) và SameSite=Lax (chống tấn công CSRF).



Hình 4.34 Kiểm tra các tiêu đề bảo mật HTTP và cờ Session Cookie qua DevTools

4.3 Bảng tổng hợp đối chứng hiệu quả an toàn Trước và Sau phòng thủ

STT	Kịch bản kiểm thử tấn công	Trạng thái Trước phòng thủ (Before)	Trạng thái Sau phòng thủ (After)	Phản ứng của Snort IDS
1	SQL Injection cơ bản (' OR 1=1 --)	Trả về toàn bộ danh mục sản phẩm do mệnh đề WHERE bị vô hiệu hóa; nguy cơ rò rỉ dữ liệu cao.	Trả về 0 kết quả (total: 0); toàn bộ payload được xem là chuỗi ký tự thuần túy thông qua Parameterized Binding.	Kích hoạt cảnh báo Rule 100001 (ALARM-SQLi Attempt).
2	Trích xuất dữ liệu (UNION SELECT)	Có thể bị ghép câu lệnh để dump toàn bộ tài khoản và mật khẩu băm trong bảng users.	Ngăn chặn triệt để việc ghép lệnh; bảo vệ toàn vẹn dữ liệu bảng người dùng.	Kích hoạt cảnh báo Rule 100002 (ALARM-SQLi UNION SELECT).
3	Dò quét tệp nhạy cảm (/.env, /.git)	Tệp cấu hình có thể bị tải về dạng plain text, làm lộ mật khẩu database và APP_KEY.	Nginx chặn đứng tại tầng Web Server, trả về mã lỗi 404 Not Found / 403 Forbidden.	Kích hoạt cảnh báo Rule 100004 (ALARM-PROBE Sensitive File).
4	Vét cạn mật khẩu (Brute Force Login)	Cho phép gửi request thử mật khẩu vô hạn lần; nguy cơ bị bẻ khóa và DoS máy chủ.	Rate Limiting chặn sau 5 lần sai, khóa tạm thời với mã HTTP 429; yêu cầu giải Math CAPTCHA.	Kích hoạt cảnh báo Rule 100005 (ALARM-BRUTEFORCE).
5	Leo thang đặc quyền (Truy cập /admin)	Người dùng thường hoặc khách vãng lai tự gõ URL vẫn có thể xem trang quản trị.	Middleware RBAC kiểm tra cờ is_admin, chặn đứng với mã lỗi HTTP 403 Forbidden.	Ghi nhận truy cập trái phép tầng ứng dụng.

STT	Kịch bản kiểm thử tấn công	Trạng thái Trước phòng thủ (Before)	Trạng thái Sau phòng thủ (After)	Phản ứng của Snort IDS
6	Tấn công tầng trình duyệt (Clickjacking, XSS)	Không có tiêu đề phòng vệ; cookie phiên dễ bị đánh cắp qua JavaScript (document.cookie).	Áp dụng 7 tiêu đề HTTP Security (CSP, nosniff, HSTS) và bật cờ HttpOnly, SameSite=Lax.	Ngăn chặn tấn công từ phía Client / Trình duyệt.

KẾT LUẬN

1. Kết luận

Qua quá trình nghiên cứu, thiết kế và triển khai thực nghiệm đề tài **"Phát triển và nâng cao an toàn bảo mật cho website bán hàng thiết bị số DDH Electronics"**, nhóm tác giả đã hoàn thành toàn diện các mục tiêu đề ra theo chuẩn chương trình môn học *Phát triển phần mềm web an toàn*:

- **Về mặt lý luận và thiết kế:** Nhóm đã hệ thống hóa và áp dụng thành công mô hình phòng thủ theo chiều sâu (Defense-in-Depth) 4 tầng, phân tích các nguy cơ tấn công trọng yếu dựa trên tiêu chuẩn quốc tế OWASP Top 10 và ánh xạ thành công các biện pháp phòng vệ tương ứng vào kiến trúc hệ thống.
- **Về mặt hiện thực hóa mã nguồn (Application Security):** Triệt tiêu hoàn toàn lỗ hổng SQL Injection bằng cơ chế Eloquent Parameterized Query; ngăn chặn tấn công vét cạn (Brute Force) bằng tổ hợp Rate Limiting và Math CAPTCHA động; kiểm soát phân quyền nghiêm ngặt theo mô hình RBAC với mã lỗi chặn 403 Forbidden; loại trừ rủi ro mã độc Web Shell (RCE) qua thuật toán tải tệp tin an toàn 5 lớp (kiểm tra phần mở rộng, phân tích mã nhị phân MIME bằng PHP Fileinfo và định danh lại tệp bằng UUID); đồng thời thiết lập trọn bộ 7 tiêu đề an toàn trình duyệt (Security Headers: CSP, HSTS, X-Frame-Options, nosniff) và bảo vệ Session Cookie với các cờ HttpOnly, SameSite=Lax.
- **Về mặt hạ tầng mạng và giám sát (Network Security):** Máy chủ Web Nginx được cấu hình tối ưu hóa bảo mật với chứng chỉ SSL/TLS 1.2 và 1.3, tự động chuyển hướng 301 sang HTTPS và khóa cấm tuyệt đối việc dò quét các tệp tin cấu hình nhạy cảm (.env, .git). Đồng thời, hệ thống phát hiện xâm nhập Snort IDS được tích hợp bộ quy tắc my_sql.rules nhận dạng chính xác và phát cảnh báo thời gian thực trước mọi hành vi thăm dò, khai thác mạng.
- **Về mặt thực nghiệm đối chứng:** Kịch bản kiểm thử tự động hóa 6 bài test thực chiến đã chứng minh sự khác biệt rõ rệt giữa hai trạng thái Trước (Before) và Sau (After) phòng thủ, khẳng định tính hiệu quả và độ tin cậy cao của các giải pháp đã triển khai.

2. Hướng phát triển của đề tài

Trong thời gian tới, nhằm nâng cao hơn nữa năng lực bảo mật và sẵn sàng cho môi trường vận hành quy mô lớn, đề tài có thể tiếp tục mở rộng theo các hướng sau:

- **Tích hợp xác thực đa yếu tố (2FA / Multi-Factor Authentication):** Triển khai giao thức mã khóa một lần dựa trên thời gian (TOTP) qua các ứng dụng như Google Authenticator / Microsoft Authenticator dành riêng cho phân hệ quản trị viên (Admin).
- **Nâng cấp hệ thống phát hiện và ngăn chặn xâm nhập (IPS & WAF):** Tích hợp tường lửa ứng dụng web chuyên sâu ModSecurity WAF và nâng cấp lên phiên bản Snort 3 với bộ tiền xử lý HTTP Inspector để tự động giải mã các kỹ thuật làm rối mã nguồn (Obfuscation: Hex/Base64 encoding).
- **Phân tán phiên làm việc và Rate Limiting trên In-Memory Cache:** Sử dụng Redis Cluster để quản lý bộ đếm giới hạn truy cập và lưu trữ Cookie phiên làm việc tập trung, phục vụ kiến trúc cân bằng tải (Load Balancing) đa máy chủ.

TÀI LIỆU THAM KHẢO

[1] **ThS. Phương Văn Cảnh**, *Bài giảng Phát triển phần mềm web an toàn*, Khoa Công nghệ Thông tin, Trường Đại học Điện lực (EPU), 2026.

[2] **Open Web Application Security Project (OWASP)**, *OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks*, https://owasp.org/Top10/.

[3] **Taylor Otwell & Laravel Core Team**, *Laravel 11.x Documentation: Security, Authentication, Middleware, and Database Query Builder*, https://laravel.com/docs/11.x.

[4] **Cisco Systems & Sourcefire**, *Snort Users Manual (Version 2.9.x) - Network Intrusion Detection & Prevention System*, https://www.snort.org/documents.

[5] **Nginx Software**, *Nginx HTTP Server Reference Architecture: ngx_http_ssl_module and Security Hardening Guide*, https://nginx.org/en/docs/.

[6] **IETF (Internet Engineering Task Force)**, *RFC 8446 - The Transport Layer Security (TLS) Protocol Version 1.3*, https://datatracker.ietf.org/doc/html/rfc8446.

[7] **Mozilla Developer Network (MDN)**, *Web Security: Content Security Policy (CSP), HTTP Headers and Cookie Security Attributes*, https://developer.mozilla.org/.